

АННОТАЦИЯ

Тема выпускной квалификационной работы — «Разработка платформы для генерации резюме с применением технологий искусственного интеллекта на основе стартапа Resugen», в рамках которой были рассмотрены современные тенденции цифровизации кадрового делопроизводства и внедрение алгоритмических систем отбора персонала (Applicant Tracking Systems).

Целью работы является создание архитектуры и минимально жизнеспособного продукта (MVP) платформы «Resugen», выступающей в роли цифрового карьерного ассистента и обеспечивающей повышение совместимости резюме с алгоритмами машинного анализа. Для достижения поставленной цели были сформулированы основные задачи:

1. Провести комплексный анализ международного и российского рынков сервисов для создания и оптимизации резюме, провести анализ аудитории.
2. Сформулировать детализированные функциональные и нефункциональные требования к разрабатываемой платформе.
3. Спроектировать архитектуру дополнительных сервисов бизнес-процесса и управления пользователем и фронтенд часть.
4. Реализовать дополнительные сервисы бизнес-процесса, управления пользователями и фронтенд часть .
5. Описать жизнеспособную бизнес-модель проекта, построить финансовую модель, оценить рыночный потенциал (TAM, SAM, SOM) и проанализировать риски.

Работа состоит из 62 страниц, включающих 11 рисунков, 1 таблицу. Для решения поставленных задач использовано 30 источников. Область применения данной работы — техническое обеспечение процессов трудоустройства.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....	2
1 ТЕОРЕТИЧЕСКИЕ ОСНОВЫ РАЗРАБОТКИ ИНТЕЛЛЕКТУАЛЬНЫХ HR-СЕРВИСОВ.....	5
1.1 Эволюция систем автоматизации рекрутмента и анализа резюме.....	5
1.2 Современные методы обработки естественного языка в HR-задачах	8
1.3. Проблематика и требования интеллектуальных сервисов поддержки при создании резюме.....	11
1.4 Обзор и классификация существующих решений.....	13
2 ПРОЕКТИРОВАНИЕ И РЕАЛИЗАЦИЯ СЕРВИСОВ УПРАВЛЕНИЯ ПОЛЬЗОВАТЕЛЯМИ И ФРОНТЕНДА.....	16
2.1 Функциональные и нефункциональные требования к разрабатываемой платформе.....	16
2.2 Проектирование архитектуры сервисов бизнес-процесса и управления пользователями.....	19
2.3 Реализация сервисов управления пользователями и аутентификации	23
2.4. Реализация дополнительных сервисов бизнес-процесса.....	27
2.5 Реализация фронтенд-части и интеграция с сервисами.....	31
3 ЭКОНОМИЧЕСКАЯ КОМПОНЕНТА И ТЕСТИРОВАНИЕ СТАРТАП-ПРОЕКТА.....	39
3.1 Тестирование MVP.....	39
3.2 Оценка эффективности платформы методом ансамблевого LLM- скоринга.....	47
3.3 Анализ сильных и слабых сторон.....	51
3.4 Оценка рыночного потенциала.....	53
ЗАКЛЮЧЕНИЕ.....	58
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....	59

ВВЕДЕНИЕ

На сегодняшний день информационные технологии развиваются с огромной скоростью и проникают во все сферы деятельности человека. Процессы, которые ранее требовали значительных временных и человеческих ресурсов, все чаще автоматизируются с помощью программных алгоритмов и систем искусственного интеллекта. Сфера управления персоналом и рекрутмента не является исключением: на смену ручному отбору и субъективной оценке приходят автоматизированные системы подбора, способные в кратчайшие сроки обрабатывать большие объемы данных.

Под влиянием цифровизации изменились и требования к первичному взаимодействию между соискателем и работодателем. Если раньше резюме оценивалось исключительно человеком, то теперь на первом этапе оно зачастую проходит через фильтры алгоритмических систем отбора. Эти системы анализируют структуру документа, наличие определенных ключевых элементов и семантическое соответствие описанию вакансии. В результате качество трудоустройства соискателя начинает напрямую зависеть не только от его профессиональных компетенций, но и от того, насколько корректно его резюме подготовлено с технической точки зрения для машинного анализа.

Исходя из этого, возникает объективная потребность в инструментах, которые помогают соискателю адаптироваться к новым технологическим реалиям рынка труда. Пользователю необходим сервис, способный не просто предоставить шаблон для заполнения, но и выступить в роли интеллектуального ассистента: проанализировать требования вакансии, дать рекомендации по улучшению содержания резюме и спрогнозировать его совместимость с автоматизированными системами отбора. Разработка подобных сервисов требует применения современных методов обработки естественного языка и проектирования сложных распределенных систем.

Актуальность работы обусловлена возрастающим разрывом между традиционными методами составления резюме и современными алгоритмическими подходами к их оценке. В условиях постоянного совершенствования технологий автоматизации рекрутинга, умение создавать конкурентоспособное и технологически совместимое резюме становится критически важным навыком. В связи с этим возникает необходимость в создании доступных и эффективных цифровых инструментов, способных снизить порог вхождения соискателей в цифровую среду кадрового отбора и повысить прозрачность этого процесса.

Целью работы является разработка архитектуры и MVP для интеллектуальной веб-платформы «Resugen», выступающей в роли «цифрового карьерного ассистента»

Для достижения поставленной цели необходимо решить следующие основные задачи работы:

1. Провести комплексный анализ международного и российского рынков сервисов для создания и оптимизации резюме, провести анализ аудитории.
2. Сформулировать детализированные функциональные и нефункциональные требования к разрабатываемой платформе.
3. Спроектировать архитектуру дополнительных сервисов бизнес-процесса и управления пользователем и фронтенд часть.
4. Реализовать дополнительные сервисы бизнес-процесса, управления пользователями и фронтенд часть .
5. Описать жизнеспособную бизнес-модель проекта, построить финансовую модель, оценить рыночный потенциал (TAM, SAM, SOM) и проанализировать риски.

Объектом исследования являются системы генерации структурированного контента.

Предметом исследования является автоматизация рекрутмента и создания резюме с использованием языковых моделей.

Новизна работы заключается в комплексном подходе к проектированию платформы, который сочетает в себе детерминированные алгоритмы структурной оптимизации документа с методами семантического анализа и генерации текста. В отличие от упрощённых конструкторов документов, предлагаемое решение ориентировано на обеспечение совместимости резюме с алгоритмическими механизмами отбора за счёт глубокой интеграции методов обработки естественного языка в процесс создания документа.

Практическая значимость работы заключается в создании готового к использованию программного инструмента, который позволяет конечным пользователям сократить временные затраты на подготовку профессиональных документов при трудоустройстве и повысить их релевантность требованиям конкретных вакансий. Разработанные в ходе работы архитектурные принципы и программные модули могут быть использованы в качестве основы для дальнейшего развития интеллектуальных ассистентов в области управления карьерой.

1 ТЕОРЕТИЧЕСКИЕ ОСНОВЫ РАЗРАБОТКИ ИНТЕЛЛЕКТУАЛЬНЫХ HR-СЕРВИСОВ

1.1 Эволюция систем автоматизации рекрутмента и анализа резюме

Интеллектуальные HR-системы возникли из классических ATS, которые с 1970-х годов трансформировались от простых баз резюме до облачных и AI-решений[1][2]. Современные ATS автоматизируют 70–80% рутинных операций рекрутера (сбор резюме, рассылки, первичный отбор)[3][4]. Согласно отраслевой статистике, порядка 75% поступающих резюме отфильтровываются системами на первых этапах[3], что значительно ускоряет процесс найма. На Рисунке 1.1.1 показана хронология развития данных систем.

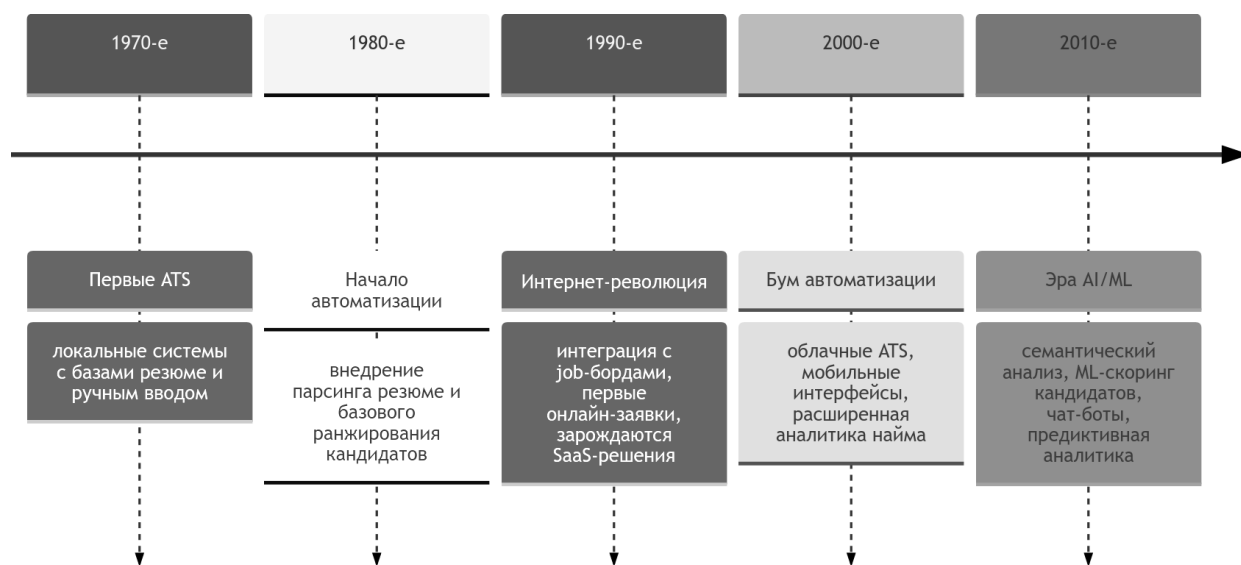


Рисунок 1.1.1 — Хронология развития ATS

Появление первых ATS датируется 1970-ми годами[1]. Эти системы представляли собой электронные хранилища резюме, где данные вводились вручную; поиск осуществлялся по простым ключевым словам. В 1980-х годах появились первые алгоритмы парсинга резюме[1] — системы автоматически выбирали из анкеты ключевые поля (образование, опыт) и сортировали кандидатов. Однако ранние ATS были громоздкими и дорогими:

позволить их внедрение могли лишь крупные корпорации. В этот период системы рекрутинга давали преимущество централизованной базе кандидатов, но чувствительность и гибкость фильтров оставляли желать лучшего.

Распространение интернета в 1990-х радикально изменило HR-инструменты[1]. Веб-сайты компаний и job-порталы (например, Monster, CareerBuilder) упростили публикацию вакансий и сбор резюме. ATS стали интегрироваться с этими ресурсами: кандидаты могли подавать заявки онлайн, а системы автоматически импортировали данные[1]. В 2000-х, благодаря облачным технологиям, ATS вышли за рамки enterprise: SaaS-модель обеспечила малым и средним предприятиям доступ к современным инструментам без капитальных затрат[1][3]. Появились мобильные приложения для рекрутеров и кандидатов, что ещё больше ускорило процесс найма. Автоматические алгоритмы анализировали сразу тысячи резюме, формируя шорт-листы по релевантности (с учётом опыта, навыков, образования)[5]. В результате время закрытия вакансий стало измеряться днями, а не неделями – показатель времени закрытия позиций существенно сократился[3][4].

Начиная с 2010-х, ATS активно используют машинное обучение и NLP[2][3]. Системы могут семантически сравнивать вакансии и резюме: например, модели BERT узнают, что «React-разработчик» и «Frontend React» эквивалентны[6]. Автоматические скоринг-функции оценивают каждого кандидата по сложным алгоритмам, учитывая десятки параметров (навыки, стаж, достижения), часто формируя единый балл согласованно с историей успешных наймов. Интеллектуальные инструменты также включают возможность прогнозирования успеха: на основе данных прошлых наймов система вычисляет вероятность успешного прохождения испытательного срока или долгосрочной эффективности кандидата. Расширенные ATS предлагают чат-ботов и виртуальных ассистентов, которые проводят предскрининг, задают первичные вопросы кандидатам и согласуют

интервью[2][7]. При этом системы адаптируются к контексту и синонимам, анализируя не просто ключевые слова, а смыслы фраз.

Современные ATS имеют преимущества:

- выполняют комплексную автоматизацию рутинных операций, таких как поиск и скрининг кандидатов, рассылка сообщений, планирование[3][4];
- ведут объединённые базы данных кандидатов с историей взаимодействий;
- предоставляют отчётность и аналитику в реальном времени по KPI найма (время на этап, источники заявок и так далее).

Это позволяет существенно повышать эффективность рекрутеров и формировать стратегию подбора на основе данных.

Однако данные системы имеют ограничения и не способны полностью заменить роль человека. Системы уязвимы к качеству исходных данных: ошибки парсинга документов из-за особенности их формата или нетривиального форматирования содержимого текста могут ухудшать результаты. Алгоритмы могут упустить нестандартные резюме и от выдающихся кандидатов, если они не соответствуют заданным настройкам. Кроме того, автоматические фильтры известны своей «жесткостью»: по статистике StandOut CV, около 75% резюме отсекаются автоматикой на первом этапе[3], что уменьшает время рекрутера, но требует контроля ошибок. Также важно учитывать этические аспекты: без должного надзора системы могут незаметно воспроизводить скрытую предвзятость в данных, включая даже гендерные, возрастные или иные смещения[2][6].

Следовательно, технологии ATS следует применять в тандеме с человеческим контролем и периодической валидацией.

1.2 Современные методы обработки естественного языка в HR-задачах

Современный рекрутмент опирается на достижения NLP для автоматизации анализа текстовой информации. Основные технологии включают распознавание сущностей, векторное встраивание текстов, семантический поиск, классификацию, а также глубокие нейронные сети (Transformer, RNN) и семантические онтологии навыков. Эти методы применяются для извлечения и сравнения информации в резюме и вакансиях, построения рекомендательных систем и интерактивных чат-ботов.

Ключевые поля из резюме и вакансий (имя, опыт, образование, навыки) выделяются методами NER (Named Entity Recognition) и синтаксического анализа. Например, современные модели умеют корректно распознавать дату и промежутки времени, наименование компаний и должностей в произвольном формате[6]. Для этого используют как классические CRF-модели, так и нейросетевые подходы (BERT/BiLSTM, spaCy, DeepPavlov и другие). На их основе формируют расширенные графы знаний: связывают кандидата с индустрией, описывают иерархию навыков. Результат – структурированная база данных из резюме и вакансий, пригодная для дальнейшего анализа.

При подборе кандидатов системы переходят от точного совпадения слов к семантической близости. Словесные представления (word2vec, fastText, SBERT) преобразуют текстовые описания в векторы. Обученные на HR-корпусах трансформеры (BERT и производные) учитывают контекст: они понимают, что «project manager» и «руководитель проектов» – синонимичные термины[12]. Это позволяет ранжировать отклики не только по вхождению ключевых слов, но и по смысловой релевантности. NLP-алгоритмы ранжирования (learning-to-rank) обучают модель отличать успешные совпадения резюме с заданной вакансией. В качестве тренировочных данных используются прошлые решения по найму (отмечено

в ACL-обзоре как перспективное направление[2]). Результат – многофакторный скоринг кандидатов, улучшающий качество отбора.

Помимо простого фильтра, современные системы применяют машинное обучение для классификации резюме по релевантности или прогнозирования поведения кандидата. Например, модели классифицируют резюме как «соответствующее» или «не соответствующее» требованиям вакансии. Также используются регрессионные алгоритмы для прогноза долгосрочности контракта и уровня успеха кандидата[2]. Часто решения представляют NLP-задачу как ранжирование: алгоритм обучается на помеченных данных – кто был приглашён на интервью, нанят или уволен – и находит оптимальные критерии оценки. Такие подходы ускоряют первичный отбор, позволяя HR-специалистам сосредоточиться на узком круге сильных кандидатов.

NLP лежит в основе интерактивных HR-ботов, которые общаются с кандидатами при помощи текста или речи. Например, чат-боты задают уточняющие вопросы соискателю, помогают заполнить форму отклика или записаться на интервью. Алгоритмы Intent Detection и Dialog Management обеспечивают понимание вопросов («Какие требования к опыту?») и поиск ответов в базе вакансий. Некоторые системы интегрированы с ASR (Automatic Speech Recognition) и TTS (Text-To-Speech), позволяя провести голосовое интервью с анализом речевых характеристик. По оценкам HR-отрасли, чат-боты ускоряют время ответа кандидату и сокращают операционные издержки на 30–50%[8], особенно при массовом найме.

Современные ATS-платформы начинают использовать LLM (Large Language Models) и генеративный ИИ: они автоматически составляют тексты вакансий, индивидуальные ответы кандидатам и даже рекомендации по улучшению резюме. Например, система может предложить переформулировать описание достижений кандидата под ключевые требования. Гибридные решения комбинируют традиционные алгоритмы с нейросетями: параллельно с BERT-классификатором вычисляют числовые метрики совпадений и объединяют их в ансамбль. Такие сложные конвейеры

улучшают точность отбора и минимизируют влияние брешей в обучении моделей.

Преимущества методов NLP:

- обеспечение масштабируемости (тысячи резюме обрабатываются за секунды);
- объективность (снижая влияние субъективности при первичном скрининге[4][8]) и возможность глубокой аналитики (выявление скрытых шаблонов успешности);
- обработка естественного языка превращает рекрутинг в Data-driven процесс, позволяя анализировать эффективность каналов поиска и качество оценок.

Тем не менее, методы NLP также ограничены, поскольку модели требовательны к данным: ошибки при извлечении (например, из PDF или изображения) оказывают значительное влияние на результат. Автоматические системы являются лишь инструментом поддержки решений, который может пропустить нетривиальных кандидатов или повторить предвзятость данных[2][6]. Требуется регулярная калибровка моделей и участие специалистов, контролирующих работу системы.

Можно сделать вывод, что систематическое применение NLP в рекрутменте стремительно расширяет возможности автоматизации данной сферы. Распределение задач между машинами и людьми становится более чётким: человек оставляет за собой стратегические решения и валидацию результатов. При этом важно помнить, что внедрение ИИ-технологий требует тщательной подготовки данных и постоянного мониторинга их работы. Интегрированный подход, сочетающий современные NLP-алгоритмы и экспертное знание бизнеса, позволяет создавать интеллектуальные HR-сервисы, способные ускорить и улучшить подбор персонала.

1.3. Проблематика и требования интеллектуальных сервисов поддержки при создании резюме

Резюме просматривают не люди, а алгоритмы, ищущие точные совпадения ключевых слов и паттернов. Это приводит к нескольким проблемным аспектам:

- **Массовая автофильтрация резюме.** Более половины заявок блокируется «на первом этапе» без участия человека[11][12]. Кандидаты не получают обратной связи о поводах для отказа (строгие фильтры по опыту, образованию или определённым терминам[13][14]).
- **Нечеловеческие предпочтения и смещения.** Алгоритмы наследуют тренды обучающих данных. Например, известный кейс Amazon 2018 показал, что ИИ отказывал женщинам (из-за смещённого обучающего набора). В современном подборе риски «black-box» решений остаются: алгоритм может отбросить резюме с обучением на определённых курсах или русскими именами без объяснения. Это требует особого контроля над искусственным интеллектом[12].
- **Ошибки форматирования и контентные пробелы.** Сложные макеты (таблицы, графика, изображения) часто «ломают» разборщики резюме[14][15]. Аналогично, использование синонимов вместо строгих терминов вакансии снижает процент попадания: ATS обычно ищут точные ключевые фразы[14]. Требуется автоматическое выявление таких проблем: например, уведомлять кандидата об обнаруженных «странных» объектах и предлагать упростить формат (одноколоночный текст, стандартные шрифты)[15].
- **Зависимость от AI и «нечеловеческая» речь.** Попытки полностью доверить написание CV генеративным моделям приводят к чрезмерно формальным или пафосным формулировкам[13][16]. Лексика выходит за привычный стиль (кандидаты используют фразы, которые люди редко говорят), что снижает доверие. С другой стороны, массовое

применение одинаковых шаблонов (например, автоматических сопроводительных писем) делает кандидатские профили однообразными[16].

- **Приватность и безопасность данных.** Резюме содержат личные данные (имена, контакты и другие), часто кандидаты загружают их в облако. Вопросы GDPR (для зарубежных компаний) и российского законодательства становятся критичными[17].

Учитывая вышеуказанные проблемы, «умный помощник» при создании резюме должен реализовывать следующий функционал:

- **Парсинг и анализ контента.** Распознавать сущности (имя, образование, опыт, навыки) с помощью NER и синтаксического анализа[18]. Автоматически выделять ключевые навыки в тексте. Проверять наличие и формат важных полей (например, даты в единообразном формате).
- **ATS-оптимизация.** Платформа должна анализировать описания вакансий и предлагать включить точные ключевые фразы из них в резюме[14]. Например, конвертировать синонимы или жаргон должности в формальные термины работодателя. Это может включать дообучение модели на локальных данных компании (fine-tuning) для лучшего понимания внутренней терминологии.
- **Улучшение текста.** Сервис выступает редактором: предлагает переформулировки для ясности и конкретики, убирает избыточные слова. Генерирует примеры достижений («увеличил продажи на X%») и добавляет релевантные метрики (например, количество подчинённых или объем проектов). Это делает резюме более информативным для алгоритмов и людей.
- **Интерактивные боты и подсказки.** Уточнение информации в интерактивных полях, вопросы про опыт, детали. Это снижает разрыв между формальным резюме и личным опытом.

1.4 Обзор и классификация существующих решений

Российский HR-рынок представлен сочетанием локальных ATS-систем, AI-конструкторов резюме и порталов. В Таблице 1.4.1 представлен список прямых и косвенных конкурентов.

Таблица 1.4.1 — Прямые и косвенные конкуренты

Продукт	Категория	Русский язык	AI-Функции	Модель
Huntflow	ATS/CRM	Да	Да	Подписка
Talantix	ATS/CRM	Да	Да	Индивидуальный план
СберПодбор	ATS/CRM	Да	Да	Freemium, от 5000 руб/месяц
CareerLab	AI-резюмемер	Нет	Да	Freemium
Rezi	AI-резюмемер	Нет	Да	
Resume.com	Конструктор	Нет	Нет	Бесплатно
Kickresume	AI-конструктор	Нет	Да	
Workday HCM	ATS/CRM	Нет	Да	Индивидуальный план
SAP SuccessFactors	ATS/CRM	Нет	Да	Индивидуальный план
Canva	Дизайн	Да	Да	Freemium
Teal	AI-резюмемер	Нет	Да	Бесплатно

Например, «ТалантЭкс» («Talantix») – ATS/CRM из экосистемы «HeadHunter» (глубокая интеграция с hh.ru и CV-парсером); «СберПодбор» – HR-платформа Сбера для массового найма с мобильным интерфейсом; «Поток» (Potok.io) – решение для рекрутинга с упором на массовый и госзаказ[19]. Эти системы оптимизированы под русскоязычные данные, предлагают стандартный функционал (автопарсинг резюме, аналитика откликов) и часто интегрируются с российскими порталами («HeadHunter», «SuperJob»)[17][20].

Среди сервисов генерации резюме и карьерных помощников: «GarmonyCV» – чат-бот-ассистент к «CareerLab» для интерактивного создания «резюме» (сам подбирает фразы, проверяет стиль). «ruGPT» –

отечественная нейросеть, заменяющая западные LLM для генерации текстов. Большинство этих сервисов позиционируются как B2C, имеют freemium-модель (базовый функционал бесплатен, расширенные возможности по подписке).

Крупнейшие площадки по трудоустройству включают встроенные резюме-редакторы. Например, «HeadHunter» бесплатно генерирует однотипное резюме по шаблону с возможностью выбора языка[B8]. «Работа.ру» предлагает обширный набор шаблонов и мультиязычный интерфейс[21].

Некоторые сервисы маскируются под HR-ассистентов. К примеру, «Headz» – приложение, где соискатели «ставят лайки» вакансии, а алгоритм находит лучшие соответствия по зарплате и стеку. При этом никакие личные данные не передаются – контакты раскрываются только после согласия обеих сторон. Во многих российских ATS появились встроенные чат-боты для первичного интервью и сбора данных (см. «ИИ-агент для первого контакта» у HH[13]).

Глобальные HR-технологии делятся на корпоративные решения и потребительские AI-сервисы:

- **Корпоративные ATS/HCM.** Лидеры – «Workday», «SAP SuccessFactors» («Taleo»), «Oracle Cloud HCM», «iCIMS», «Greenhouse», «SmartRecruiters», «Bullhorn» и др. Эти платформы поддерживают многочисленные интеграции, аналитику и часто имеют функции на основе искусственного интеллекта (ранжирование кандидатов, предсказание эффективности). По оценкам аналитиков, мировой рынок ATS растёт ~7–10% в год[17], достигая к 2025 г. нескольких миллиардов долларов (консервативная оценка – \$2.47 млрд в 2025[17]). Большинство таких систем ориентированы на англоязычный сегмент, но предлагают мультиязычные интерфейсы.
- **Онлайн-конструкторы резюме.** «CareerLab» – «экосистема» для создания резюме с AI-анализом вакансий, адаптацией под каждую

позицию, подсказками экспертов и гарантией ATS-совместимости[21]; В мировой практике популярны «Kickresume» – гибридный сервис с интеллектуальным редактором и большим количеством шаблонов[11]; «Resume.com» – бесплатный конструктор с сотнями шаблонов, ориентированный на рекрутеров из крупных корпораций[21]; «Enhancv» – креативный конструктор с продвинутыми подсказками (имеются ограничения бесплатного периода)[21]; «Rezi» – платформа, полностью автоматизирующая создание CV при помощи языковых моделей[21]. Также широко используются сервисы «Teal», «Jobscan», «MyPerfectResume», «VisualCV» и «Canva» (предлагает шаблоны, но без гарантии ATS-совместимости[11]). Многие из этих инструментов генерируют не только резюме, но и сопроводительные письма на основе введенных данных и актуальных вакансий.

Таким образом, российский рынок характеризуется сосуществованием специализированных локальных ATS/CRM-систем (Huntflow, Talantix, СберПодбор), глубоко интегрированных с отечественными порталами по трудоустройству и оптимизированных под русскоязычный контент, и широкого спектра международных AI-сервисов для создания и адаптации резюме (CareerLab, Kickresume, Rezi и др.), при этом как глобальные корпоративные платформы (Workday, SAP), так и российские игроки активно внедряют искусственный интеллект для автоматизации рекрутинговых процессов, предлагая как B2B-подписки, так и freemium-модели для конечных соискателей.

2 ПРОЕКТИРОВАНИЕ И РЕАЛИЗАЦИЯ СЕРВИСОВ УПРАВЛЕНИЯ ПОЛЬЗОВАТЕЛЯМИ И ФРОНТЕНДА

2.1 Функциональные и нефункциональные требования к разрабатываемой платформе

Разрабатываемая платформа Resugen представляет собой комплексный веб-сервис, предназначенный для автоматизированной генерации и оптимизации резюме с учётом требований современных систем автоматического отбора. Основной функциональной задачей платформы является устранение разрыва между субъективными представлениями соискателей о качественном резюме и объективными алгоритмическими критериями, предъявляемыми к документам автоматизированными системами. Для достижения этой цели система должна обеспечивать управление пользователями, возможность создания и редактирования анкет, генерацию и хранение резюме, работу с вакансиями, а также предоставление аналитических инструментов для оценки качества создаваемых документов.

Система должна поддерживать полноценный цикл управления пользователями, включая регистрацию и авторизацию, с подтверждением учётной записи через электронную почту, а также предоставлять возможность восстановления доступа при утере пароля. Пользовательский аккаунт служит основой для последующей работы с анкетой, резюме и вакансиями, а также хранения истории изменений и аналитических отчётов.

Ключевым элементом платформы является работа с анкетами. Анкета формирует основу для генерации резюме и должна включать все необходимые данные о пользователе: персональные сведения, образование, опыт работы, профессиональные навыки и достижения. Платформа

обеспечивает проверку полноты и корректности введенной информации, в том числе валидацию обязательных полей.

Процесс работы с резюме организован таким образом, чтобы пользователь мог получить документ, полностью оптимизированный под требования конкретной вакансии. Генерация резюме осуществляется на основе данных анкеты и предоставленного пользователем описания вакансии. Система автоматически формирует структуру документа, соответствующую стандартам ATS, обеспечивая корректное чтение и парсинг системой работодателя. Кроме того, для повышения релевантности текста резюме используется семантический анализ и адаптация содержания с помощью больших языковых моделей. Такой подход обеспечивает не только техническую корректность, но и улучшает соответствие резюме ожиданиям работодателя, повышая вероятность прохождения первичного отбора. На Рисунке 2.1.1 представлен основной процесс работы платформы.

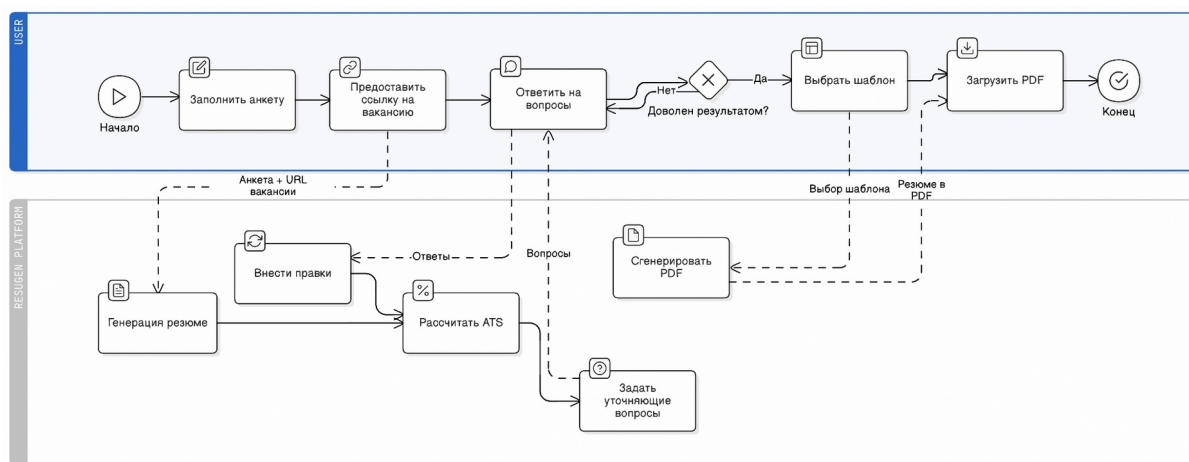


Рисунок 2.1.1 — Основной бизнес-процесс

Платформа также реализует возможности работы с вакансиями, включая импорт и сохранение вакансий из внешних источников, таких как hh.ru. Система извлекает ключевые требования и компетенции, указанные в вакансии, что позволяет автоматически адаптировать резюме под конкретную позицию. Пользователь получает возможность создавать, редактировать и удалять вакансии, хранить их в личном кабинете, а также повторно использовать для генерации новых резюме. Такая

функциональность обеспечивает полное управление процессом подготовки документов и сокращает время на анализ требований.

Аналитическая составляющая платформы включает предоставление подробных отчётов по каждому резюме. Пользователь получает ATS-оценку, отражающую соответствие документа требованиям системы автоматического отбора, а также выявление сильных и слабых сторон текста. Сервис генерирует конкретные рекомендации по улучшению содержания и структуры резюме, обеспечивая прозрачность процесса и предоставляя пользователю полный контроль над финальным документом. Аналитические данные представлены в удобном визуальном формате, что позволяет быстро ориентироваться в результатах и принимать решения о доработке резюме.

Нефункциональные требования к платформе направлены на обеспечение производительности, надёжности, безопасности, масштабируемости и удобства использования. В частности, время отклика сервиса при генерации резюме не должно превышать 5–10 минут даже при обработке больших анкет и сложных вакансий. Система должна обеспечивать сохранность данных пользователей, а также логирование всех операций для возможности аудита. Архитектура платформы разрабатывается с учётом горизонтального масштабирования сервисов, что позволяет поддерживать рост числа пользователей без снижения производительности. Безопасность данных достигается шифрованием на уровне базы данных и передачи по сети, а также защитой от распространённых атак, таких как SQL-инъекции, XSS и CSRF.

Особое внимание уделяется модульности архитектуры и поддержке расширяемости. Разделение системы на отдельные сервисы бизнес-процессов, аналитики и фронтенд обеспечивает возможность интеграции новых алгоритмов обработки текста, языковых моделей и функциональных модулей без необходимости полной переработки платформы. Интерфейс

пользователя разрабатывается с учётом современных требований интерфейса, обеспечивая интуитивную навигацию, совместимость с различными устройствами и минимизацию количества действий для достижения результата. Это позволяет пользователю эффективно и быстро создавать ATS-оптимизированное резюме с высокой степенью релевантности вакансии, экономя время и когнитивные ресурсы.

Таким образом, функциональные и нефункциональные требования формируют комплексную основу для проектирования платформы Resugen, обеспечивая баланс между технологической корректностью, аналитической точностью, безопасностью и удобством использования. Реализация этих требований позволит создать инструмент, способный повысить эффективность взаимодействия соискателя с современными системами автоматизированного отбора и обеспечить прозрачность процесса подготовки профессиональных резюме.

2.2 Проектирование архитектуры сервисов бизнес-процесса и управления пользователями

В соответствии с требованиями, которые были подробно описаны в предыдущей главе, при проектировании необходимо уделить особое внимание таким параметрам, как масштабируемость и надежность архитектуры. Наиболее подходящим выбором можно считать микросервисную архитектуру, поскольку она позволит применить большое количество приемов, направленных на повышение отказоустойчивости, поддерживаемости и расширяемости платформы, а также упростит командную разработку отдельных компонентов системы.

Для минимально жизнеспособного продукта будет разработана базовая микросервисная архитектура с применением технологий Docker и Docker Compose для упаковки и развертывания сервисов приложения в виде контейнеров. На данный момент каждый сервис будет иметь только один

экземпляр, но в дальнейшем такую архитектуру будет возможно масштабировать и разделять на кластеры, используя Kubernetes, а также оптимизировать нагрузку при помощи балансировщиков трафика, адаптируя платформу для работы с нарастающим количеством пользователей.

На Рисунке 2.2.1 представлена схема архитектуры платформы, включающая сервисы бизнес-логики, аналитики, маршрутизации, базы данных и хранилища, а также интеграции со сторонними платформами.

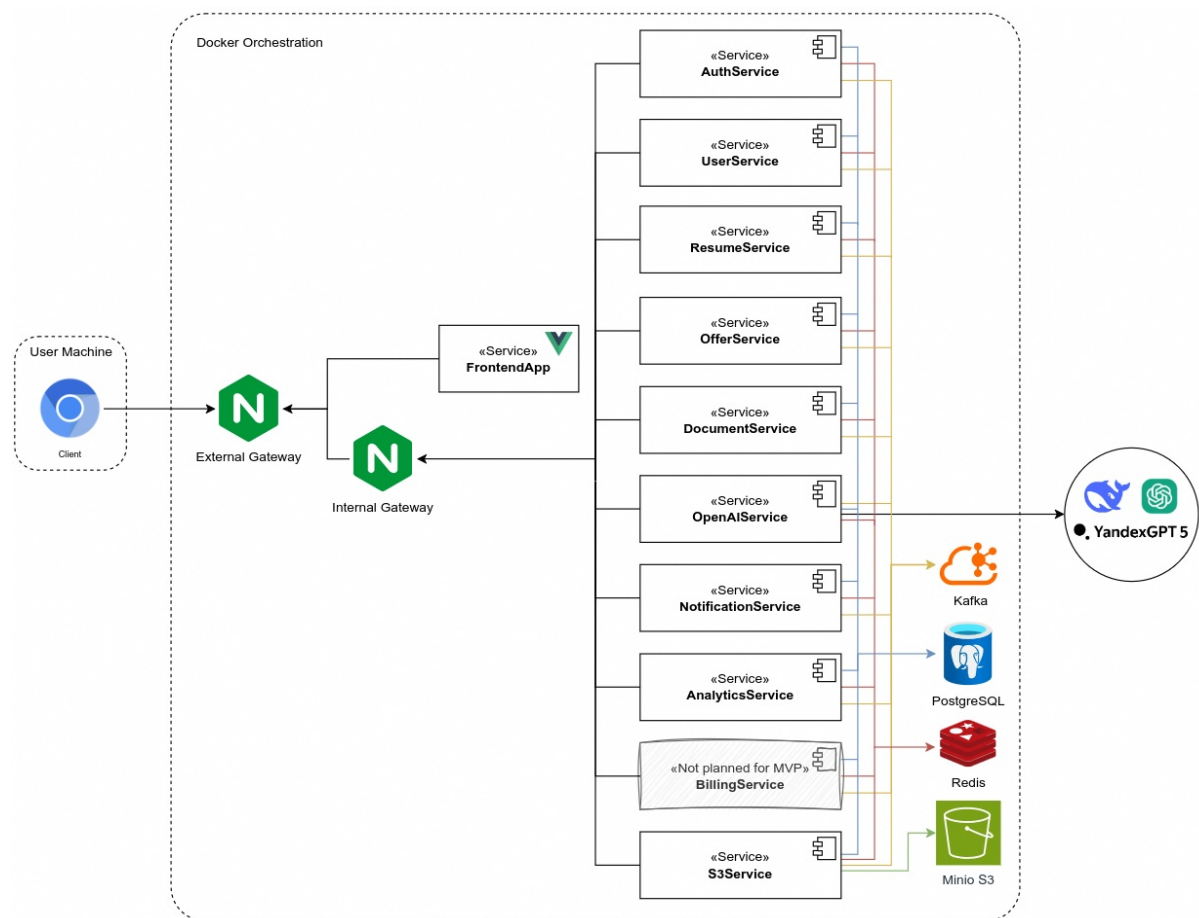


Рисунок 2.2.1 — Микросервисная архитектура платформы

Среди реализуемых согласно заданию сервисов можно выделить следующие:

- **AuthService** (сервис аутентификации и авторизации) – отвечает за регистрацию пользователей, выдачу и валидацию JWT-токенов, управление сессиями и восстановление пароля. Взаимодействует с UserService для получения профильной информации.

- ***UserService*** – управляет пользовательскими анкетами: хранение персональных данных, образования, опыта работы, навыков. Предоставляет API для создания, редактирования и получения анкеты. Данные сохраняются в реляционной базе PostgreSQL. Сервис также отвечает за валидацию и проверку полноты анкетных данных.

- ***OfferService*** – отвечает за импорт и хранение вакансий из внешних источников (например, hh.ru) или вручную введенных пользователем. Извлекает ключевые требования, компетенции и ключевые слова, которые затем используются для адаптации резюме. Данные вакансий хранятся в PostgreSQL, поддерживается полнотекстовый поиск.

- ***OpenAIService*** – сервис-адаптер к большим языковым моделям, таким как DeepSeek и ChatGPT. Получает текст резюме и требования вакансии, возвращает оптимизированную версию текста с улучшенной релевантностью. Для повышения производительности результаты кэшируются в Redis при повторных запросах с теми же входными данными.

- ***S3Service*** – обеспечивает унифицированный доступ к объектному хранилищу (Minio S3) для всех сервисов, инкапсулируя операции загрузки, скачивания и генерации временных ссылок.

Все сервисы упакованы в Docker-контейнеры и управляются с помощью оркестратора (Docker Compose на этапе разработки, в продуктивной среде планируется использовать Kubernetes). Представлены внутренний и внешний шлюзы Nginx, которые занимаются проксированием запросов. Внутренний шлюз обеспечивает коммуникацию между микросервисами, а внешний шлюз связывает фронтенд-приложение с бекендом, предоставляя доступ только к разрешенным для пользователя маршрутам. Он также обеспечивает аутентификацию, ограничение частоты запросов и защиту соединения с пользователем при помощи TLS.

Для постоянного хранения данных анкет, пользователей, сессий и вакансий используется база данных PostgreSQL. В соответствии с заданием на Рисунке 2.2.2 представлены таблицы для сервисов пользователей и вакансий. Данные пользователя разделены на отдельные таблицы для безопасности и улучшения масштабируемости. Для полей имени, электронной почты, хеша пароля и IP-адреса предусмотрены типы данных с оптимальными размерами для сокращения потребляемого дискового пространства. Таблицы анкет и вакансий хранят информацию, требуемую для генерации оптимальных резюме и в основном содержат текстовые поля для поддержки более гибких данных. Для хранения файлов документов используется S3-совместимое хранилище Minio.

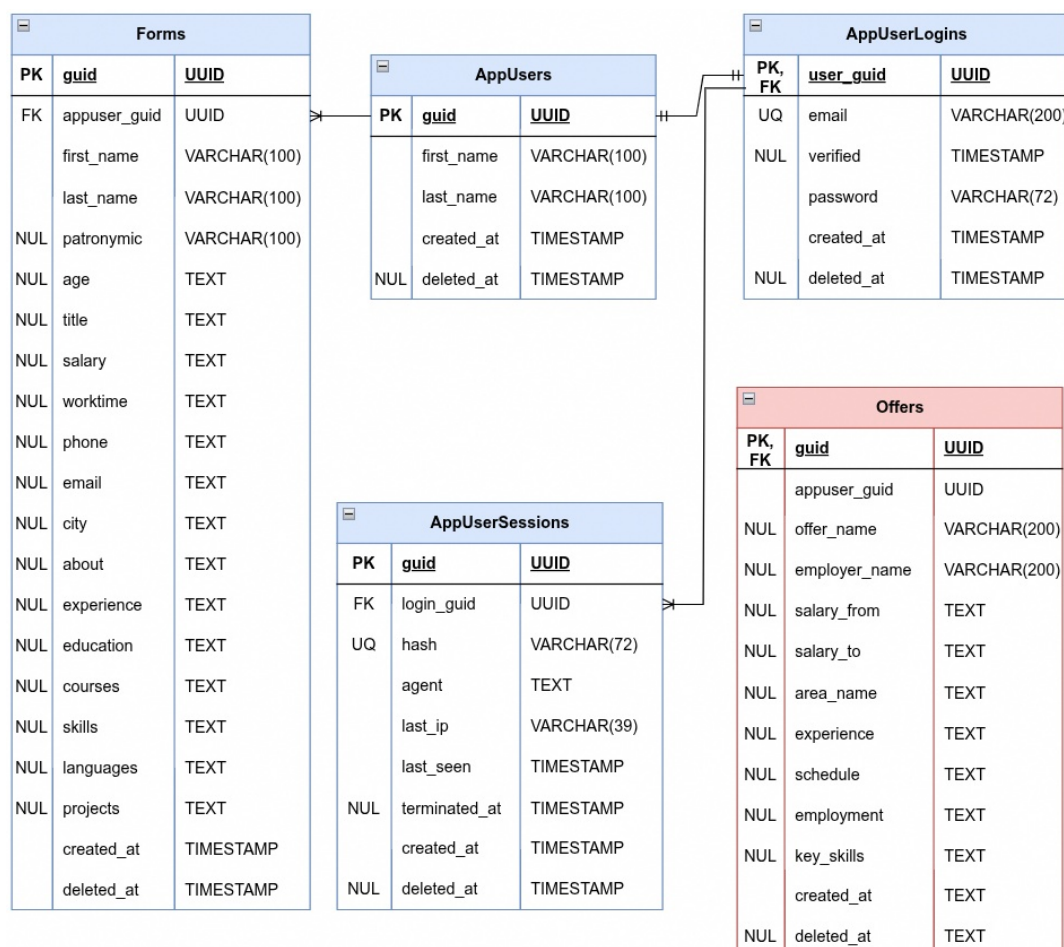


Рисунок 2.2.2 — Таблицы баз данных UserService и OfferService

2.3 Реализация сервисов управления пользователями и аутентификации

Разрабатываемому приложению требуется выполнять анкетирование пользователя с указанием большого спектра персональных данных для генерации резюме. Это делает чрезвычайно важным реализацию системы аутентификации с тщательным вниманием к безопасности и защите.

Алгоритм аутентификации действует следующим образом: при помощи маршрута «POST /login» пользователь предоставляет свои данные для входа, среди которых предусмотрены электронная почта и пароль (в дальнейшем будет возможна реализация поддержки сторонних платформ в качестве провайдеров аутентификации). Вычисляется bcrypt-хеш пароля и сверяется с хешем, который хранится в базе для данного пользователя. Если пароль валиден, то создается новая сессия и генерируется пара из токенов доступа (access) и обновления (refresh): первый предоставляет краткосрочный доступ к ресурсу, а второй имеет длительный срок действия и позволяет получать новые токены доступа по мере их истечения (при помощи маршрута «POST /refresh»).

Далее в Листинге 2.3.1 представлен фрагмент кода с методом входа в приложение.

Листинг 2.3.1 — Фрагмент кода метода входа

```
func (a *authServiceImpl) Login(
    cinfo dto.ClientInfo, request models.LoginRequest,
) (*models.LoginResponse, error) {
    var userRow database.User // структура пользователя в базе данных
    var err error
    time.Sleep(utils.GenerateRandomDelay())
    helper, db, err := database.NewDbHelperTransaction(a.dbctx)
    if err != nil {
        a.log.Error("Failed to open a transaction", zap.Error(err))
        return nil, errs.ErrServerError
    }
    defer helper.RollbackOnError(err)
    // 1. Получение пользователя по электронной почте
    userRow, err = db.TXQueries.GetUserByEmail(db.CTX, request.Email)
    if err != nil {
        var returnedError error
        if errors.Is(err, pgx.ErrNoRows) {
            returnedError = errs.ErrForbidden
        } else {

```



```
        returnedError = errs.ErrServerError
    }
    a.log.Warn("Failed login attempt: user not found",
        zap.String("email", request.Email),
        zap.Error(err))
    return nil, returnedError
}
err = utils.VerifyPassword(userRow.PasswordHash, request.Password)
if err != nil {
    a.log.Warn("Failed login attempt: invalid password",
        zap.String("email", request.Email))
    return nil, errs.ErrForbidden
}
session, err := a.registerSession(context.TODO(), userRow.ID, cinfo,
&db.TXQueries)
if err != nil {
    a.log.Error("Failed to register new session",
        zap.Error(err))
    return nil, errs.ErrServerError
}
err = a.redisClient.SetSession(context.TODO(), session.Guid.String(),
session)
if err != nil {
    a.log.Error("Failed to cache session in Redis",
        zap.Error(err))
}
accessToken, refreshToken, err := utils.GenerateTokens(
    userRow.Email,
    session.Guid.String(),
    enums.Role(userRow.Role),
)
if err != nil {
    a.log.Error("Failed to generate tokens for a new login",
        zap.String("email", userRow.Email),
        zap.Error(err))
    return nil, errs.ErrServerError
}
if err = helper.Commit(); err != nil {
    a.log.Error("Failed to commit transaction",
        zap.Error(err))
    return nil, errs.ErrServerError
}
response := models.LoginResponse{
    Tokens: models.Tokens{
        AccessToken: accessToken,
        RefreshToken: refreshToken,
    },
}
return &response, nil }
```

Важно отметить, что каждый отдельный вход в приложение привязывается к сессии с уникальным идентификатором и метаданными. Незнакомые сессии возможно прервать в личном кабинете, девалидируя все привязанные к ним JWT-токены.

Полный алгоритм аутентификации представлен на диаграмме последовательности на Рисунке 2.3.2.

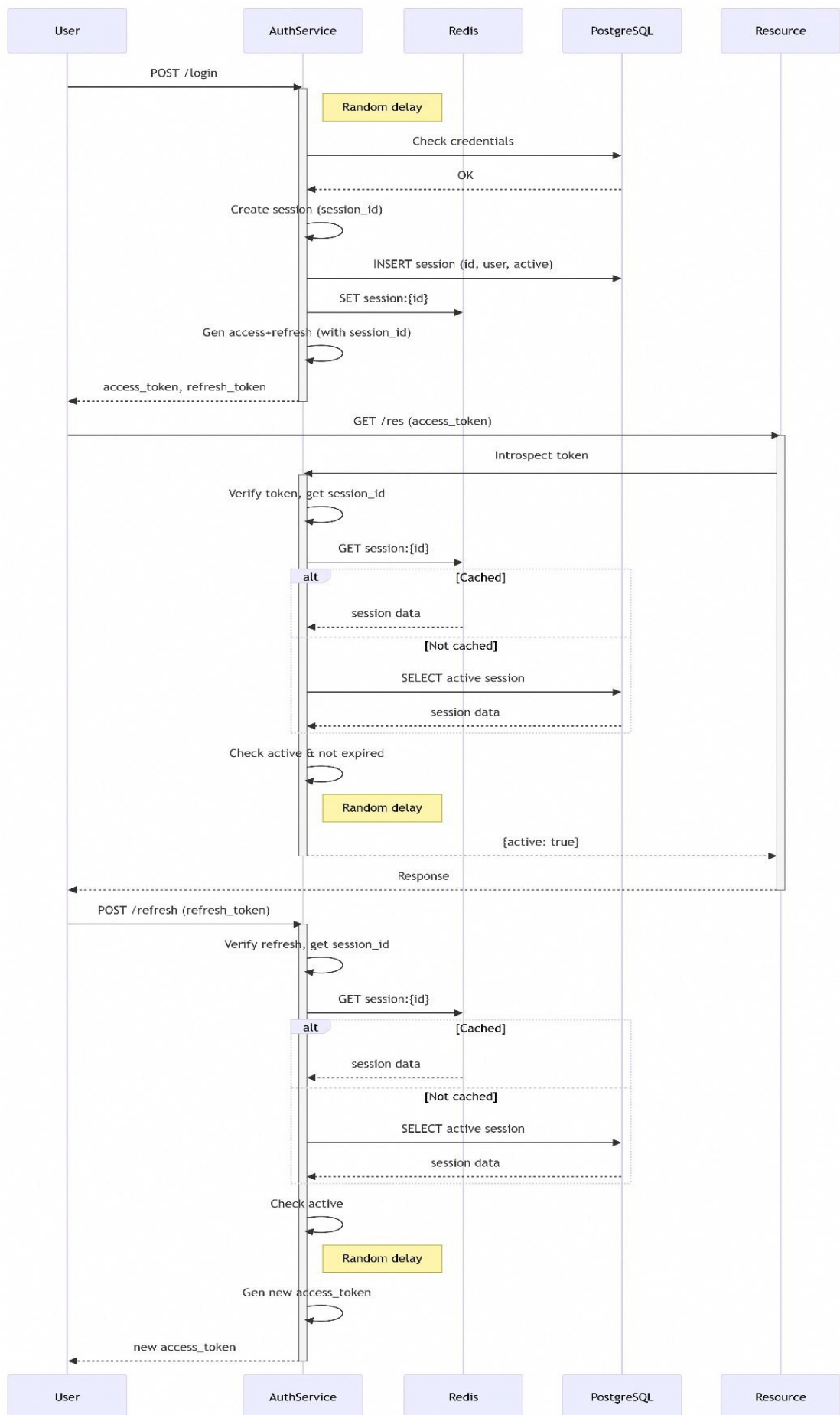


Рисунок 2.3.2 — Алгоритм аутентификации в приложении

Обобщая, можно сделать вывод, что предложенная система аутентификации будет защищать пользователя от несанкционированного доступа при компрометации учётных данных или токенов, от атак перебора паролей и эnumерации пользователей (за счёт случайных задержек), от использования перехваченных access-токенов при их коротком сроке жизни, а также от долгосрочного захвата сессии злоумышленником благодаря возможности пользователя самостоятельно завершить подозрительные сессии и мгновенно девалидировать все связанные с ними JWT-токены.

Регистрация нового пользователя на платформе Resugen реализована в сервисе AuthService и включает обязательное подтверждение учётной записи через электронную почту. При создании аккаунта проверяется уникальность адреса, пароль хешируется алгоритмом bcrypt, после чего в базе данных UserService создаётся запись с минимальным набором атрибутов. На указанный email отправляется верификационный токен с ограниченным сроком действия; переход по ссылке активирует учётную запись и предоставляет доступ к заполнению анкеты.

Процедура сброса пароля инициируется через маршрут восстановления доступа, доступный неавторизованным пользователям. После ввода зарегистрированного email система генерирует одноразовый токен сброса, который передаётся по защищённому каналу электронной почты. Пользователь переходит по ссылке с этим токеном, устанавливает новый пароль, после чего все активные сессии данного аккаунта принудительно завершаются, а соответствующие JWT-токены помещаются в чёрный список Redis. Такой подход предотвращает сохранение доступа у злоумышленника в случае компрометации учётной записи до смены пароля.

Создание и управление анкетой полностью делегировано сервису UserService. После успешной аутентификации пользователь получает доступ к API анкетирования, где может последовательно заполнить разделы, охватывающие персональные сведения, образование, опыт работы, навыки и достижения. Сервис выполняет обязательную валидацию полноты данных,

проверяя наличие критически важных полей, требуемых для последующей генерации резюме. Все изменения сохраняются в реляционных таблицах PostgreSQL, а история правок версионизируется для возможности отката. Данные анкеты изолированы от аутентификационных метаданных, что соответствует требованиям безопасности и масштабируемости микросервисной архитектуры платформы.

2.4. Реализация дополнительных сервисов бизнес-процесса

В соответствии с ранее описанной микросервисной архитектурой платформы Resugen, ключевыми компонентами, обеспечивающими интеллектуальную адаптацию резюме под требования конкретной вакансии, являются сервисы «OpenAIService» и «OfferService». Первый из них выступает в роли универсального адаптера к большим языковым моделям, в частности «DeepSeek» и «ChatGPT», а второй отвечает за получение, хранение и предварительную обработку данных о вакансиях из внешних источников, таких как hh.ru. Совместная работа этих сервисов позволяет автоматизировать процесс оптимизации текста резюме, повышая его релевантность и соответствие критериям автоматизированных систем отбора.

Сервис «OpenAIService» спроектирован как изолированный микросервис, предоставляющий REST API для выполнения запросов на преобразование текста резюме с учётом требований вакансии. Его основная задача заключается в инкапсуляции логики взаимодействия с провайдерами языковых моделей, унификации форматов запросов и ответов, а также в обеспечении отказоустойчивости и производительности за счёт механизмов кэширования. При получении запроса сервис формирует промпт, содержащий исходный текст резюме, перечень ключевых требований и компетенций, извлечённых из описания вакансии, а также специальные инструкции по улучшению стиля, устранению избыточности и выделению релевантных достижений. Сформированный промпт направляется к выбранной модели (по умолчанию используется DeepSeek-V3,

демонстрирующая высокое качество генерации текстов на русском языке при меньшей стоимости по сравнению с аналогами). В целях оптимизации повторяющихся запросов в архитектуру OpenAIService интегрирован слой кэширования на базе Redis. Ключом кэша выступает хеш-сумма, вычисленная на основе конкатенации входных данных (текст резюме и требования вакансии). При совпадении хеша сервис возвращает ранее сгенерированный ответ без обращения к внешней модели, что значительно сокращает задержки и экономит вычислительные ресурсы. В случае отсутствия данных в кэше выполняется HTTP-запрос к API провайдера, ответ сохраняется в Redis с заданным временем жизни, после чего возвращается клиенту. Для повышения надёжности реализована обработка ошибок сети и ограничений частоты запросов (rate limiting) с повторными попытками с экспоненциальной задержкой. OpenAIService также логирует все запросы и ответы в деперсонализированном виде для последующего анализа качества генерации и возможной дообучения моделей.

В Листинге 2.4.1 представлен метод генерации одного сообщения.

Листинг 2.4.1 — Метод обработки одного сообщения в OpenAIService

```
func (s *OpenAIService) processSinglePrompt(ctx context.Context, prompt
Prompt) GenerationResult {
    logger := s.logger.With(
        zap.String("prompt_id", prompt.ID),
        zap.String("model", string(prompt.Model)),
    )

    cacheKey := s.buildCacheKey(prompt)
    cached, err := s.redis.Get(ctx, cacheKey).Result()
    if err == nil {
        logger.Debug("Cache hit for prompt")
        return GenerationResult{
            PromptID: prompt.ID,
            Text:      cached,
            Cached:    true,
        }
    } else if err != redis.Nil {
        logger.Warn("Redis error during cache get", zap.Error(err))
        // Продолжаем без кэша.
    }

    logger.Info("Cache miss, requesting LLM provider")

    generatedText, err := s.generateWithRetry(ctx, prompt)
```

```
    if err != nil {
        logger.Error("LLM generation failed after retries", zap.Error(err))
        return GenerationResult{
            PromptID: prompt.ID,
            Error:     err.Error(),
        }
    }

    if err := s.redis.Set(ctx, cacheKey, generatedText, 24*time.Hour).Err(); err != nil {
        logger.Warn("Failed to cache generation result in Redis", zap.Error(err))
    }

    return GenerationResult{
        PromptID: prompt.ID,
        Text:      generatedText,
        Cached:    false,
    }
}
```

Параллельно с OpenAIService функционирует OfferService, предназначенный для управления данными о вакансиях. Ключевая функциональность сервиса заключается в автоматизированном импорте вакансий с платформы hh.ru по прямой ссылке, предоставленной пользователем. Типичный URL имеет вид «<https://api.hh.ru/vacancies/12345>», из которого извлекается числовой идентификатор вакансии. Сервис выполняет авторизованный GET-запрос к публичному API hh.ru по адресу «<https://api.hh.ru/vacancies/{id}>», получая в ответ структурированный JSON-объект, содержащий полное описание вакансии, как показано в Листинге 2.4.2. Для соблюдения правил использования API и предотвращения блокировок OfferService включает настраиваемый HTTP-клиент с указанием корректного User-Agent, а также механизмы ограничения частоты запросов. Из полученных данных извлекается и сохраняется в реляционной таблице PostgreSQL вся необходимая информация, при этом особое внимание уделяется полям, используемым для последующего семантического анализа и адаптации резюме: названию должности, списку требований и профессиональных компетенций. Для поддержки быстрого поиска по тексту описания создаётся полнотекстовый индекс. Кроме импорта из внешних источников, OfferService предоставляет REST API для ручного создания,

редактирования и удаления вакансий, что позволяет пользователям работать с позициями, опубликованными на других площадках или полученными из альтернативных каналов.

Листинг 2.4.2 — Получение данных вакансии в сервисе OfferService при помощи API hh.ru

```
func (s *OfferService) fetchFromHH(ctx context.Context, vacancyID string)
(*hhVacancyResponse, error) {

    if err := s.rateLimiter.Wait(ctx); err != nil {
        return nil, fmt.Errorf("rate limiter wait: %w", err)
    }

    apiURL := fmt.Sprintf("https://api.hh.ru/vacancies/%s", vacancyID)
    req, err := http.NewRequestWithContext(ctx, http.MethodGet, apiURL, nil)
    if err != nil {
        return nil, err
    }

    req.Header.Set("User-Agent", "Resugen/1.0 (contact@resugen.ru)")

    resp, err := s.httpClient.Do(req)
    if err != nil {
        return nil, fmt.Errorf("http request failed: %w", err)
    }
    defer resp.Body.Close()

    if resp.StatusCode != http.StatusOK {
        body, _ := io.ReadAll(resp.Body)
        return nil, fmt.Errorf("hh.ru API returned %d: %s",
resp.StatusCode, string(body))
    }

    var hhResp hhVacancyResponse
    if err := json.NewDecoder(resp.Body).Decode(&hhResp); err != nil {
        return nil, fmt.Errorf("failed to decode response: %w", err)
    }
    return &hhResp, nil
}
```

Взаимодействие между OpenAIService и OfferService организовано через внутренний API-шлюз Nginx, обеспечивающий маршрутизацию запросов между микросервисами. При инициации процесса генерации резюме пользовательский запрос, содержащий идентификатор анкеты и идентификатор вакансии, направляется в оркестрирующий сервис бизнес-логики (ResumeService, условно объединяющий этапы генерации). Этот сервис последовательно обращается к UserService для получения полных данных анкеты, к OfferService для извлечения описания и требований вакансии, а затем вызывает OpenAIService, передавая агрегированные

данные. После получения оптимизированного текста ResumeService формирует финальный документ в формате PDF и сохраняет его в объектном хранилище MinIO через S3Service. Все этапы процесса логируются, а результаты сохраняются в истории генераций пользователя. Такой подход обеспечивает чёткое разделение ответственности между сервисами, упрощает масштабирование и позволяет независимо обновлять алгоритмы обработки текста или механизмы интеграции с внешними платформами.

Реализация OpenAIService и OfferService в полной мере соответствует нефункциональным требованиям, заложенным в архитектуру платформы. Использование кэширования и асинхронного взаимодействия обеспечивает приемлемое время отклика при генерации резюме (менее 10 минут даже для больших объёмов данных), а модульная структура упрощает замену провайдера языковых моделей или добавление новых источников вакансий без модификации других компонентов системы. Безопасность передачи данных гарантируется использованием протокола TLS, а хранение персональных данных осуществляется в изолированной базе UserService. Таким образом, совместная работа сервисов OpenAIService и OfferService формирует технологический фундамент для автоматизированной генерации ATS-оптимизированных резюме, отвечающих высоким стандартам качества и релевантности.

2.5 Реализация фронтенд-части и интеграция с сервисами

Клиентская часть платформы Resugen реализована в виде одностраничного приложения на основе JavaScript-фреймворка Vue 3 с использованием Composition API. Выбор данной технологической связки обусловлен необходимостью построения реактивного, производительного и легко поддерживаемого пользовательского интерфейса. Для управления состоянием приложения задействовано хранилище Pinia, которое обеспечивает централизованное хранение данных аутентификации, анкет

пользователя, списков вакансий и метаданных о результатах генерации, а также синхронизацию этого состояния с локальным хранилищем браузера. Маршрутизация между страницами осуществляется посредством официальной библиотеки Vue Router в режиме HTML5 History, что позволяет формировать URL-адреса без перезагрузки страницы. Визуальное оформление выполнено с применением утилитарного CSS-фреймворка Tailwind CSS, благодаря чему удалось создать единообразный минималистичный интерфейс в светлой цветовой гамме.

Взаимодействие фронтенд-приложения с серверной частью организовано через REST API, предоставляемый внешним шлюзом Nginx. Все HTTP-запросы централизованы в экземпляре HTTP-клиента Axios, который автоматически добавляет к исходящим сообщениям заголовок авторизации «Authorization: Bearer token», извлекаемый из локального хранилища после успешной аутентификации. Для обработки ошибок сети и серверных ответов с кодами «4xx» и «5xx» реализованы глобальные перехватчики, обеспечивающие единообразное отображение уведомлений и, при необходимости, автоматическое перенаправление на страницу входа при истечении срока действия JWT-токена доступа.

Пользовательский путь начинается с прохождения процедуры регистрации или аутентификации. Соответствующие формы, представленные на Рисунке 2.5.1, реализованы с клиентской валидацией вводимых данных, включая проверку формата электронной почты и минимальной сложности пароля. При отправке формы регистрации фронтенд выполняет POST-запрос к эндпоинту `/api/auth/register` сервиса AuthService. После успешного создания учётной записи пользователь перенаправляется на страницу входа, где, введя свои учётные данные, инициирует запрос к `/api/auth/login`. В ответ сервис аутентификации возвращает пару JWT-токенов (access и refresh), которые сохраняются в защищённом хранилище браузера и используются для всех последующих авторизованных обращений к API. Дополнительно фронтенд

получает базовую информацию о профиле пользователя, которая отображается в шапке приложения и личном кабинете.

Регистрация
Создайте аккаунт для доступа к Resugen

Имя
Иван

Фамилия
Иванов

Email
ivan@example.com

Пароль
.....

Подтвердите пароль
.....

[Зарегистрироваться](#)

Уже есть аккаунт? [Войти](#)

Рисунок 2.5.1 — Страница регистрации

После успешной авторизации пользователь попадает в личный кабинет, общий вид которого показан на Рисунке 2.5.2. Кабинет спроектирован как единая панель управления, объединяющая три основных функциональных раздела: «Анкеты», «Вакансии» и «Резюме». Навигация между ними реализована в виде вкладок, каждая из которых содержит перечень соответствующих сущностей в формате карточек. Для ускорения выполнения типовых операций в верхней части кабинета размещены кнопки быстрых действий, позволяющие незамедлительно перейти к заполнению новой анкеты, добавлению вакансии или запуску процесса генерации текста резюме. При первоначальной загрузке кабинета фронтенд параллельно отправляет запросы к сервисам `UserService (/api/users/me/forms)` и `OfferService (/api/offers)`, получая списки сохранённых анкет и вакансий, которые затем отображаются в соответствующих вкладках.

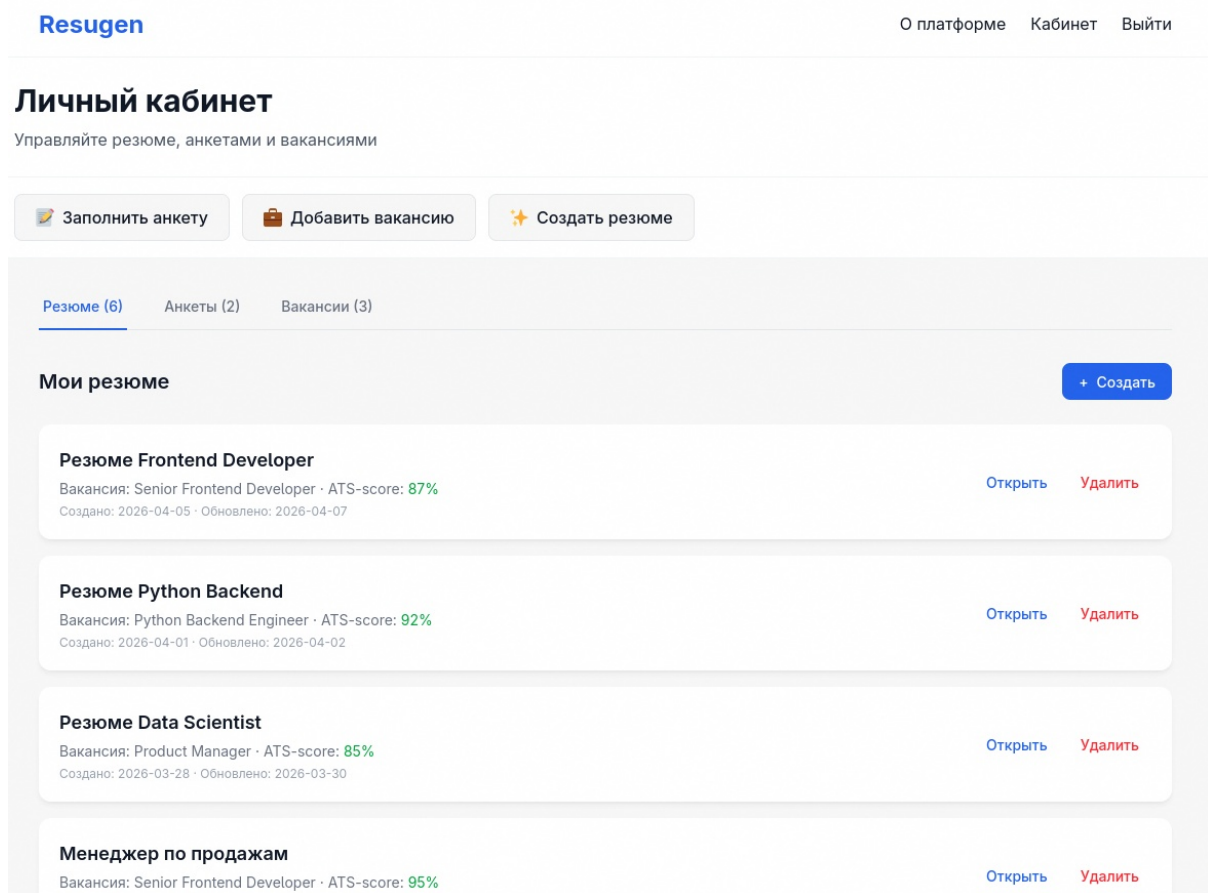


Рисунок 2.5.2 — Страница личного кабинета

Работа с анкетой представляет собой многоэтапный процесс ввода обширного набора персональных и профессиональных сведений. Интерфейс страницы создания и редактирования анкеты, изображённый на рисунке 2.5.3, содержит поля для ввода фамилии, имени, отчества, контактных данных, желаемой должности, информации об образовании, опыте работы, ключевых навыках, знании языков и реализованных проектах. Фронтенд выполняет предварительную валидацию обязательных полей, а при сохранении отправляет PUT- или POST-запрос к API UserService (/api/users/me/form). Сервис, в свою очередь, проверяет полноту данных и возвращает обновлённый объект анкеты, который сохраняется в хранилище Pinia и немедленно отображается в интерфейсе. Для обеспечения целостности данных все изменения фиксируются с указанием временной метки последнего обновления.

Новая анкета

Заполните все необходимые поля. Поля, отмеченные *, обязательны.

Название анкеты *

Например: Основная анкета

Описание

Краткое описание анкеты

Имя *

Иван

Фамилия *

Иванов

Отчество

Иванович

Возраст

30

Желаемая должность *

Frontend Developer

Желаемая зарплата

200000

График работы

Полный день

Рисунок 2.5.3 — Страница добавления анкеты

Управление вакансиями реализовано аналогичным образом, однако ключевой особенностью является интеграция с сервисом OfferService для автоматического импорта описаний с платформы hh.ru. На странице добавления вакансии пользователь может вставить прямую ссылку на интересующую его позицию. Фронтенд извлекает из URL числовой идентификатор и отправляет запрос к эндпоинту `/api/offers/import`, передавая его в теле сообщения. Сервис OfferService выполняет обращение к публичному API hh.ru, получает структурированную информацию о вакансии и возвращает её клиенту. Полученные данные автоматически заполняют соответствующие поля формы (название должности, работодатель, требования, ключевые навыки и т.д.), как показано на рисунке 2.5.4.

Пользователь имеет возможность отредактировать предзаполненные сведения или сохранить вакансию в исходном виде для последующего использования при генерации текста резюме.

Новая вакансия

Заполните информацию о вакансии. Поля со * обязательны.

Название вакансии *

Ассистент ветеринарного врача

Работодатель *

Ветеринарная клиника Оберег

Зарплата от

30000

Зарплата до

85000

Город / регион

Москва

Требуемый опыт

Без опыта

График работы

Полный день

Тип занятости

Полная занятость

Рисунок 2.5.4 — Страница добавления вакансии

Также была реализована страница настроек аккаунта, представленная на Рисунке 2.5.5. Она позволяет пользователю управлять своей учетной записью. Функционал включает в себя редактирование имени и фамилии, смену пароля, а также просмотр и прекращение активных сессий в аккаунте.

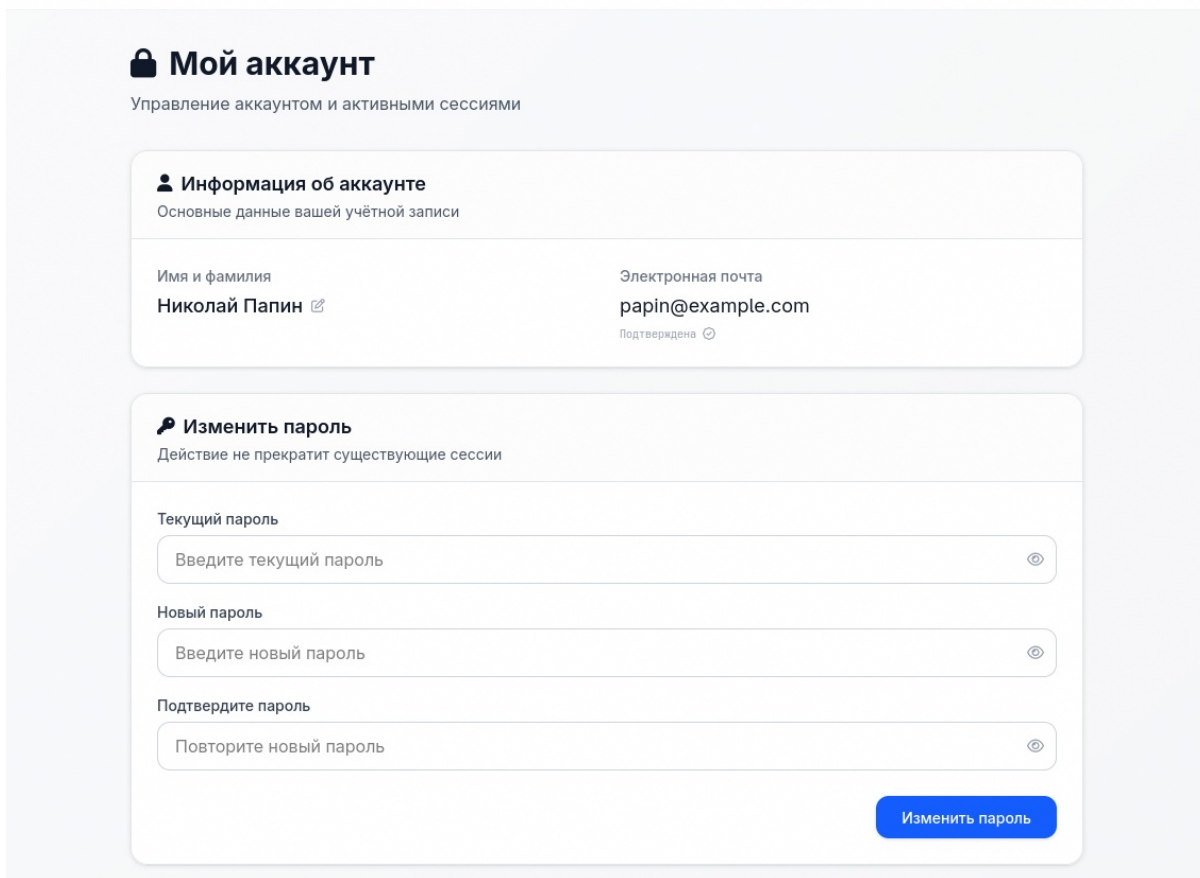


Рисунок 2.5.5 — Страница управления аккаунтом

Для изменения имени и фамилии пользователю требуется нажать на иконку редактирования в соответствующей области. Поля для смены пароля находятся в блоке ниже и предоставляют возможность временно включить отображение вводимых символов при помощи нажатия на иконку глаза, расположенную в правой части поля.

Блок управления сессиями находится под блоком изменения пароля и представлен на Рисунке 2.5.6. Он позволяет просматривать полный список активных сессий с их подробным описанием, включающее в себя название устройства, версию браузера, операционную систему, IP-адрес, агент и последнее время активности. Любую сессию, кроме текущей, возможно прервать при помощи кнопки «Завершить», расположенной справа.

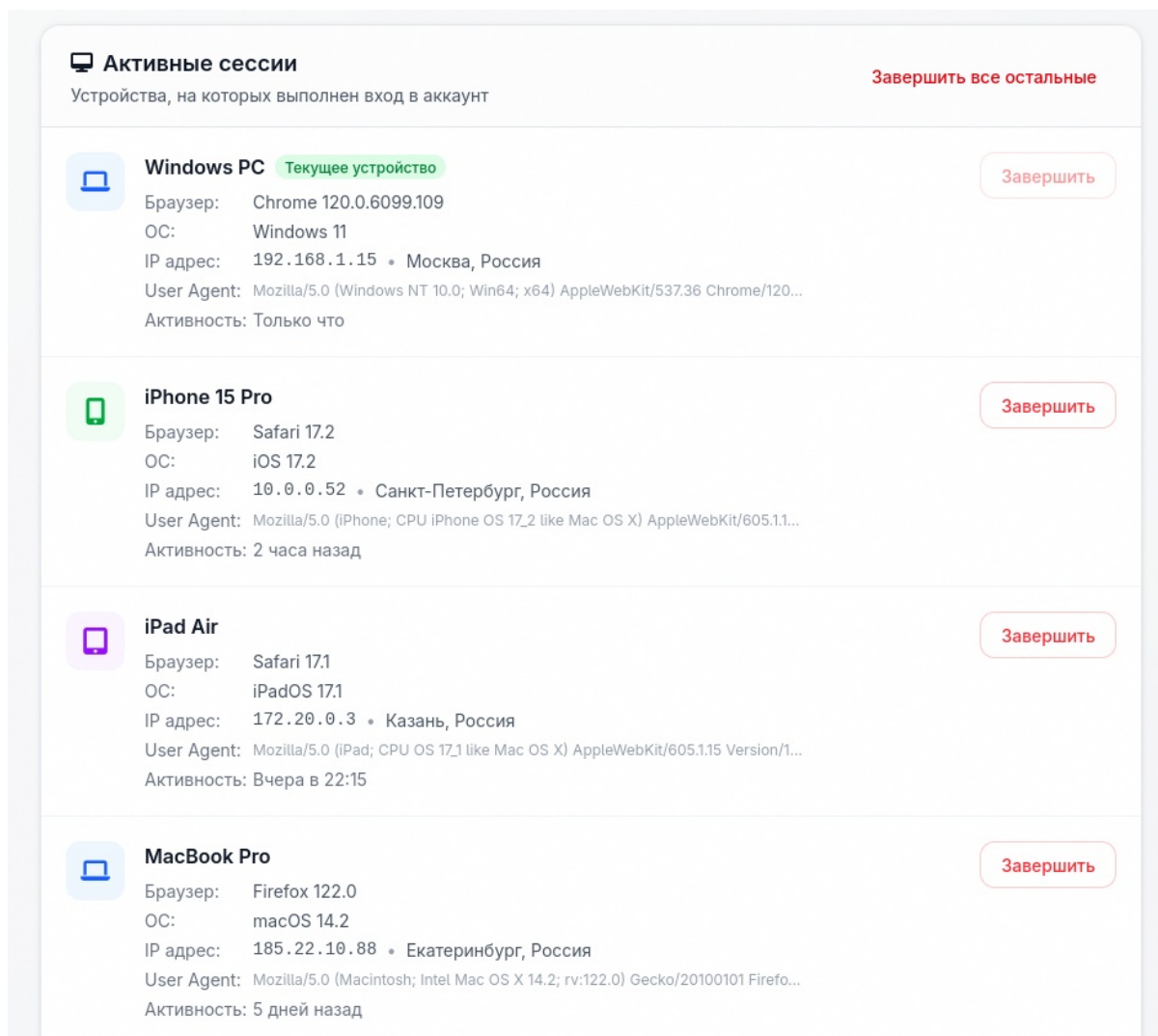


Рисунок 2.5.6 — Фрагмент страницы аккаунта со списком сессий

Таким образом, реализованная фронтенд-часть платформы Resugen в соответствии с заданием обеспечивает полноценный, интуитивно понятный интерфейс для взаимодействия пользователя с сервисами управления пользователями и дополнительными сервисами бизнес-процесса, среди которых на фронтенде были представлены сервис аутентификации и авторизации, сервис пользователей и сервис вакансий.

3 ЭКОНОМИЧЕСКАЯ КОМПОНЕНТА И ТЕСТИРОВАНИЕ СТАРТАП-ПРОЕКТА

3.1 Тестирование MVP

Обеспечение корректности и надёжности функционирования ключевых компонентов платформы Resugen выступает в качестве критического условия успешной реализации и последующей эксплуатации системы. В соответствии с архитектурными принципами, изложенными во второй главе, MVP по заданию предусматривает реализацию четырех микросервисов, среди которых есть сервисы аутентификации и авторизации, пользователя, вакансий, LLM API и S3-хранилища. Для тестирования корректной работоспособности каждого микросервиса будет использоваться методология модульного тестирования. Данный подход обеспечивает возможность изолированной проверки бизнес-логики сервисов при условии устранения зависимостей от внешних по отношению к модулю системных ресурсов, включая системы управления базами данных, брокеры сообщений и сторонние программные интерфейсы приложений. В настоящем разделе приводится систематизированное описание ключевых тестовых сценариев для каждого сервиса, характеризуется инструментальный стек, используемый в процессе верификации, а также представлены эталонные примеры реализации тестовых процедур на языке программирования Go с применением библиотек «testify» и «gomock».

Сервис AuthService содержит критически важную логику входа в систему, регистрации, управления сессиями и выпуска JWT-токенов. Тестирование метода входа, фрагмент которого приведён в листинге 2.3.1, требует изоляции от базы данных PostgreSQL, Redis и генератора токенов. В тестовом окружении используются моки интерфейсов DBQuerier (операции с БД), RedisClient (кэширование сессий) и TokenGenerator (создание JWT).

Листинг 3.1.1 демонстрирует набор тестов для успешного и неуспешного ВХОДА.

Листинг 3.1.1 — Модульные тесты метода Login сервиса AuthService

```
package auth_test

func TestAuthService_Login(t *testing.T) {
    ctrl := gomock.NewController(t)
    defer ctrl.Finish()

    mockDB := mocks.NewMockDBQuerier(ctrl)
    mockRedis := mocks.NewMockRedisClient(ctrl)
    mockTokenGen := mocks.NewMockTokenGenerator(ctrl)
    logger := zap.NewNop()

    service := auth.NewAuthService(mockDB, mockRedis, mockTokenGen, logger)

    validEmail := "user@example.com"
    validPassword := "correctPassword"
    validHash, _ := utils.HashPassword(validPassword) // предполагаемая хеш-
функция
    userID := "550e8400-e29b-41d4-a716-446655440000"
    sessionGUID := "123e4567-e89b-12d3-a456-426614174000"

    t.Run("Successful login", func(t *testing.T) {
        req := models.LoginRequest{Email: validEmail, Password:
validPassword}
        clientInfo := dto.ClientInfo{IP: "192.168.1.1", UserAgent: "test-
agent"}

        // Ожидаемые вызовы
        mockDB.EXPECT().
            GetUserByEmail(gomock.Any(), validEmail).
            Return(database.User{
                ID:            userID,
                Email:          validEmail,
                PasswordHash: validHash,
                Role:            "user",
            }, nil)

        mockDB.EXPECT().
            CreateSession(gomock.Any(), gomock.Any()).
            DoAndReturn(func(ctx context.Context, session database.Session)
error {
                assert.Equal(t, userID, session.UserID)
                assert.Equal(t, clientInfo.IP, session.IP)
                return nil
            })

        mockRedis.EXPECT().
            SetSession(gomock.Any(), gomock.Any(), gomock.Any()).
            Return(nil)

        mockTokenGen.EXPECT().
            GenerateTokens(validEmail, gomock.Any(), enums.RoleUser).
            Return("access-token", "refresh-token", nil)

        // Выполнение тестируемого метода
        resp, err := service.Login(clientInfo, req)
```

```
require.NoError(t, err)
assert.NotNil(t, resp)
assert.Equal(t, "access-token", resp.Tokens.AccessToken)
assert.Equal(t, "refresh-token", resp.Tokens.RefreshToken)
})

t.Run("User not found", func(t *testing.T) {
    req := models.LoginRequest{Email: "unknown@example.com", Password:
"any"}
    clientInfo := dto.ClientInfo{}

    mockDB.EXPECT().
        GetUserByEmail(gomock.Any(), req.Email).
        Return(database.User{}, pgx.ErrNoRows)

    resp, err := service.Login(clientInfo, req)

    assert.Nil(t, resp)
    assert.ErrorIs(t, err, errs.ErrForbidden)
})

t.Run("Invalid password", func(t *testing.T) {
    req := models.LoginRequest{Email: validEmail, Password:
"wrongPassword"}
    clientInfo := dto.ClientInfo{}

    mockDB.EXPECT().
        GetUserByEmail(gomock.Any(), validEmail).
        Return(database.User{
            ID:          userID,
            Email:        validEmail,
            PasswordHash: validHash,
            Role:         "user",
        }, nil)

    resp, err := service.Login(clientInfo, req)

    assert.Nil(t, resp)
    assert.ErrorIs(t, err, errs.ErrForbidden)
})
}
```

Дополнительные тесты покрывают процедуру регистрации с проверкой уникальности email, отправку письма подтверждения (мок SMTP-клиента), а также механизм обновления токенов с валидацией refresh-токена и завершением сессий.

Сервис OfferService взаимодействует с внешним API hh.ru и сохраняет полученные данные в PostgreSQL. Модульные тесты изолируют HTTP-клиент и репозиторий базы данных. Листинг 3.1.2 показывает тест метода fetchFromHH, проверяющий корректную обработку ответа API и соблюдение rate limiting.

Листинг 3.1.2 — Тестирование получения данных вакансии с hh.ru

```
func TestOfferService_fetchFromHH(t *testing.T) {
    ctrl := gomock.NewController(t)
    defer ctrl.Finish()

    mockHTTPClient := mocks.NewMockHTTPClient(ctrl)
    mockRateLimiter := mocks.NewMockRateLimiter(ctrl)
    logger := zap.NewNop()

    service := offer.NewOfferService(mockHTTPClient, mockRateLimiter, nil,
    logger)

    vacancyID := "123456"
    expectedURL := "https://api.hh.ru/vacancies/123456"

    t.Run("Successful fetch", func(t *testing.T) {
        mockRateLimiter.EXPECT().Wait(gomock.Any()).Return(nil)

        mockResponse := &http.Response{
            StatusCode: http.StatusOK,
            Body:
io.NopCloser(strings.NewReader("{\"id\":\"123456\",\"name\":\"Go Developer\"}")),
        }
        mockHTTPClient.EXPECT().
            Do(gomock.Any()).
            DoAndReturn(func(req *http.Request) (*http.Response, error) {
                assert.Equal(t, expectedURL, req.URL.String())
                assert.Equal(t, "Resugen/1.0 (contact@resugen.ru)",
req.Header.Get("User-Agent"))
                return mockResponse, nil
            })

        result, err := service.fetchFromHH(context.Background(), vacancyID)

        require.NoError(t, err)
        assert.Equal(t, "123456", result.ID)
        assert.Equal(t, "Go Developer", result.Name)
    })

    t.Run("API returns non-200 status", func(t *testing.T) {
        mockRateLimiter.EXPECT().Wait(gomock.Any()).Return(nil)

        mockResponse := &http.Response{
            StatusCode: http.StatusNotFound,
            Body:
                io.NopCloser(strings.NewReader("not found")),
        }
        mockHTTPClient.EXPECT().Do(gomock.Any()).Return(mockResponse, nil)

        result, err := service.fetchFromHH(context.Background(), vacancyID)

        assert.Nil(t, result)
        assert.ErrorContains(t, err, "hh.ru API returned 404")
    })
}
```

Тесты метода `ImportVacancy` дополнительно проверяют сохранение извлечённых данных в репозиторий, дедупликацию вакансий и обработку сетевых таймаутов.

Сервис OpenAIService использует Redis для кэширования результатов и HTTP-клиент для обращения к провайдеру языковых моделей. Ключевой метод processSinglePrompt (листинг 2.4.1) тестируется с моками Redis и генератора текста (мок HTTP-клиента, имитирующего ответ от LLM API). Листинг 3.1.3 содержит тесты попадания в кэш, промаха и успешной генерации, а также обработки ошибок.

Листинг 3.1.3 — Модульные тесты метода processSinglePrompt

```
func TestOpenAIService_processSinglePrompt(t *testing.T) {
    ctrl := gomock.NewController(t)
    defer ctrl.Finish()

    mockRedis := mocks.NewMockRedisClient(ctrl)
    mockLLMClient := mocks.NewMockLLMClient(ctrl)
    logger := zap.NewNop()

    service := openai.NewOpenAIService(mockRedis, mockLLMClient, logger)

    prompt := openai.Prompt{
        ID:      "prompt-1",
        Model:   openai.ModelDeepSeek,
        Text:    "Опыт работы: 5 лет разработки на Go...",
    }
    cacheKey := service.BuildCacheKey(prompt) // предполагаемый метод
    generatedText := "Оптимизированный текст резюме..."

    t.Run("Cache hit", func(t *testing.T) {
        mockRedis.EXPECT().
            Get(gomock.Any(), cacheKey).
            Return(redis.NewStringResult(generatedText, nil))

        result := service.processSinglePrompt(context.Background(), prompt)

        assert.Equal(t, prompt.ID, result.PromptID)
        assert.Equal(t, generatedText, result.Text)
        assert.True(t, result.Cached)
        assert.Empty(t, result.Error)
    })

    t.Run("Cache miss and successful generation", func(t *testing.T) {
        mockRedis.EXPECT().Get(gomock.Any(),
            cacheKey).Return(redis.NewStringResult("", redis.Nil))

        mockLLMClient.EXPECT().
            Generate(gomock.Any(), prompt).
            Return(generatedText, nil)

        mockRedis.EXPECT().
            Set(gomock.Any(), cacheKey, generatedText, 24*time.Hour).
            Return(redis.NewStatusResult("OK", nil))

        result := service.processSinglePrompt(context.Background(), prompt)

        assert.Equal(t, prompt.ID, result.PromptID)
        assert.Equal(t, generatedText, result.Text)
    })
}
```

Продолжение Листинга 3.1.3 — Модульные тесты метода `processSinglePrompt`

```
        assert.False(t, result.Cached)
        assert.Empty(t, result.Error)
    })

    t.Run("LLM generation fails", func(t *testing.T) {
        mockRedis.EXPECT().Get(gomock.Any(),
            cacheKey).Return(redis.NewStringResult("", redis.Nil))

        mockLLMClient.EXPECT().
            Generate(gomock.Any(), prompt).
            Return("", errors.New("provider rate limit exceeded"))

        result := service.processSinglePrompt(context.Background(), prompt)

        assert.Equal(t, prompt.ID, result.PromptID)
        assert.Empty(t, result.Text)
        assert.False(t, result.Cached)
        assert.Contains(t, result.Error, "rate limit")
    })
}
```

В полной версии тестов также проверяется логика повторных попыток с экспоненциальной задержкой (метод `generateWithRetry`), использующая настраиваемый `backoff`.

Сервис `UserService` отвечает за CRUD-операции с анкетами пользователей. Модульные тесты проверяют валидацию полей (обязательность образования, опыта работы), взаимодействие с репозиторием PostgreSQL и изоляцию от других сервисов. Листинг 3.1.4 демонстрирует тест создания новой анкеты с проверкой обязательных полей.

Листинг 3.1.4 — Тестирование создания анкеты в `UserService`

```
func TestUserService_CreateProfile(t *testing.T) {
    ctrl := gomock.NewController(t)
    defer ctrl.Finish()

    mockRepo := mocks.NewMockProfileRepository(ctrl)
    logger := zap.NewNop()

    service := user.NewUserService(mockRepo, logger)

    userID := "user-123"
    validProfile := models.Profile{
        UserID:      userID,
        FirstName:    "Иван",
        LastName:     "Иванов",
        Education:    "МГУ, 2020",
        Experience:   []models.WorkExperience{{Company: "ООО Ромашка",
            Position: "Разработчик", Years: 3}},
        Skills:       []string{"Go", "PostgreSQL"},
    }

    t.Run("Successful creation", func(t *testing.T) {
```

```
mockRepo.EXPECT().
    Create(gomock.Any(), gomock.Any()).
    DoAndReturn(func(ctx context.Context, p models.Profile) error {
        assert.Equal(t, validProfile.UserID, p.UserID)
        assert.NotEmpty(t, p.ID) // ID должен быть сгенерирован
        return nil
    })

    created, err := service.CreateProfile(context.Background(),
validProfile)

    require.NoError(t, err)
    assert.NotEmpty(t, created.ID)
    assert.Equal(t, validProfile.FirstName, created.FirstName)
})

t.Run("Validation fails: missing required field", func(t *testing.T) {
    invalidProfile := validProfile
    invalidProfile.Education = "" // образование обязательно

    created, err := service.CreateProfile(context.Background(),
invalidProfile)

    assert.Nil(t, created)
    assert.ErrorIs(t, err, errs.ErrValidation)
    assert.Contains(t, err.Error(), "education")
})
}
```

Дополнительно покрываются сценарии обновления анкеты с проверкой прав доступа, получения анкеты по идентификатору, а также версионирования изменений.

Сервис S3Service инкапсулирует работу с Minio SDK. Тесты используют мок клиента Minio для проверки загрузки файлов, генерации предподписанных URL и обработки ошибок хранилища. Листинг 3.1.5 показывает тест метода UploadFile.

Листинг 3.1.5 — Тестирование загрузки файла в S3-хранилище

```
func TestS3Service_UploadFile(t *testing.T) {
    ctrl := gomock.NewController(t)
    defer ctrl.Finish()

    mockMinioClient := mocks.NewMockMinioClient(ctrl)
    logger := zap.NewNop()

    service := s3.NewS3Service(mockMinioClient, "resugen-bucket", logger)

    objectKey := "resumes/user123/resume.pdf"
    fileContent := []byte("%PDF-1.4 ...")

    t.Run("Successful upload", func(t *testing.T) {
        mockMinioClient.EXPECT().
            PutObject(
```

```
        gomock.Any(),
        "resugen-bucket",
        objectKey,
        gomock.Any(),
        int64(len(fileContent)),
        minio.PutObjectOptions{ContentType: "application/pdf"},
    ).
    Return(minio.UploadInfo{Size: int64(len(fileContent))}, nil)

    err := service.UploadFile(context.Background(), objectKey,
bytes.NewReader(fileContent), "application/pdf")

    assert.NoError(t, err)
})

t.Run("Bucket does not exist", func(t *testing.T) {
    mockMinioClient.EXPECT().
        PutObject(gomock.Any(), gomock.Any(), gomock.Any(), gomock.Any(),
gomock.Any(), gomock.Any()).
        Return(minio.UploadInfo{}, minio.ErrorResponse{Code:
"NoSuchBucket"})

    err := service.UploadFile(context.Background(), objectKey,
bytes.NewReader(fileContent), "application/pdf")

    assert.Error(t, err)
    assert.Contains(t, err.Error(), "bucket")
})
}
```

Все описанные модульные тесты выполняются автоматически при каждом коммите в репозиторий с использованием GitHub Actions. Для изоляции тестовой среды применяется утилита `docker-compose.test.yml`, поднимающая контейнеры PostgreSQL и Redis исключительно для интеграционных тестов, в то время как модульные тесты используют сгенерированные моки. Покрывание кода контролируется утилитой «go test -cover», целевой показатель составляет не менее 80% для критической бизнес-логики.

Представленные примеры демонстрируют, что выбранный стек (testify и gomock) позволяет эффективно изолировать зависимости микросервисов и проверять корректность реализации в соответствии с архитектурными требованиями, изложенными во второй главе. Модульное тестирование подтверждает надёжность ключевых компонентов платформы Resugen на этапе разработки MVP и создаёт основу для последующего регрессионного тестирования при масштабировании системы.

3.2 Оценка эффективности платформы методом ансамблевого LLM-скоринга

Для проверки ключевой гипотезы о способности платформы Resugen повышать релевантность резюме требованиям современных ATS была разработана и применена методика сравнительного тестирования, в которой роль экспертных оценщиков выполняет ансамбль из трёх больших языковых моделей. Такой подход позволяет получить объективную и многомерную оценку качества документов, избегая зависимости от особенностей какой-либо одной модели и, что не менее важно, снижая эффект само-предвзятости, возникающий при оценке моделью собственных генераций.

Методология эксперимента опирается на концепцию «LLM-as-a-Judge»[22]. Согласно этой концепции, современные языковые модели способны имитировать экспертную оценку, близкую к логике принятия решений в интеллектуальных системах отбора персонала[23][24]. Учитывая известные ограничения, связанные с вариативностью и возможной предвзятостью отдельных моделей, в исследовании задействован ансамбль из трёх независимых провайдеров: Yandex GPT 5.1 Pro, DeepSeek-V3.2 и GPT-5 mini от OpenAI. Разнообразие архитектур и обучающих данных способствует взаимной компенсации систематических ошибок и повышает надёжность итоговой оценки. Для получения статистически устойчивых результатов процедура скоринга каждой пары «резюме–вакансия» выполнялась троекратно с последующим усреднением как на уровне отдельных моделей, так и по всему ансамблю. Такая схема измерений позволяет сгладить влияние стохастической природы генеративных моделей.

Экспериментальная выборка была сформирована на основе открытого датасета резюме, собранных с платформы hh.ru (<https://github.com/brom83/hh-resume-eda>). Датасет содержит более 44 тысяч записей, каждая из которых представляет собой структурированную информацию о соискателе, включающую пол, возраст, ожидаемый уровень заработной платы, желаемую

должность, город проживания, готовность к переезду и командировкам, предпочтительные формы занятости и график работы, подробное описание опыта с указанием мест работы и должностей, сведения об образовании и дату обновления резюме. Пример записи демонстрирует высокую степень детализации профессионального пути кандидата, что позволяет использовать эти данные для реалистичного моделирования процесса отбора.

Из общего массива были отобраны три группы резюме, соответствующие трём различным вакансиям, охватывающим ключевые сегменты IT-рынка и разные уровни квалификации. Первая вакансия — младший веб-разработчик (фронтенд) — ориентирована на начинающих специалистов, владеющих основами HTML, CSS и JavaScript, знакомых с одним из современных фреймворков (React, Vue или Angular), имеющих представление о принципах адаптивной вёрстки и кроссбраузерной совместимости. Вторая вакансия — аналитик данных — предполагает наличие у кандидата опыта работы с SQL на уровне сложных запросов, уверенного владения Python для анализа данных (библиотеки Pandas, NumPy), навыков визуализации в Tableau или Power BI, а также понимания основ математической статистики и A/B-тестирования. Третья вакансия — старший системный администратор — требует глубоких знаний операционных систем семейства Linux и Windows Server, опыта администрирования сетевой инфраструктуры, настройки систем мониторинга, владения скриптовыми языками и понимания принципов информационной безопасности.

Для каждой из трёх вакансий было отобрано по 10 резюме, что в сумме составило 30 экспериментальных единиц. На первом этапе каждое резюме в исходном виде (сырой текст, приведённый к базовой структуре) оценивалось ансамблем моделей. На втором этапе те же резюме обрабатывались платформой Resugen: на основе исходного текста и описания соответствующей вакансии генерировалась оптимизированная версия. Важно отметить, что для генерации оптимизированных текстов использовалась

исключительно модель DeepSeek-V3.2, тогда как оценка выполнялась полным ансамблем, включающим и саму DeepSeek-V3.2, и Yandex GPT 5.1 Pro, и GPT-5 mini. Таким образом, модель-генератор выступала лишь одним из трёх равноправных экспертов, что существенно снижает риск необъективного завышения баллов.

Для автоматизации процесса сбора оценок был скрипт на языке Python, представленный в Листинге 3.2.1, который осуществляет последовательные вызовы API трёх провайдеров с заданной повторностью. Он принимает на вход CSV-файл, содержащий пары «резюме–вакансия», и возвращает среднюю оценку релевантности по всему датасету. Процедура скоринга для каждой пары проводилась по единому стандартизированному промпту, в котором моделям предлагалось выступить в роли эксперта по подбору персонала и современной ATS, оценивая релевантность резюме требованиям вакансии по шкале от 0 до 100 баллов. При оценке учитывались семантическая близость формулировок, полнота отражения ключевых компетенций, контекстуальная уместность описания опыта и общая пригодность структуры документа для автоматизированного парсинга. Для унификации извлечения числовой оценки в промпт была включена инструкция завершать ответ фразой «Результат оценки: X». Итоговое среднее значение вычислялось путём деления суммы всех полученных баллов на общее количество измерений (30 пар умножить на 3 модели, умножить на 3 повтора, что равно 270 измерениям для каждого датасета). Скрипт запускался дважды: первый раз с файлом исходных резюме, второй раз — с файлом оптимизированных резюме. Для ускорения работы скрипт выполняет обработку тестовых данных асинхронно у всех трех провайдеров одновременно.

Листинг 3.2.1 — Программа вычисления средней оценки качества резюме

```
import os
import re
import asyncio
import sys
import pandas as pd
from openai import AsyncOpenAI

YANDEX_API_KEY = os.getenv("YANDEX_API_KEY")
DEEPSEEK_API_KEY = os.getenv("DEEPSEEK_API_KEY")
OPENAI_API_KEY = os.getenv("OPENAI_API_KEY")

PROVIDERS = {
    "yandex": {
        "client": AsyncOpenAI(api_key=YANDEX_API_KEY,
base_url="https://api.yandex.com/v1/"),
        "model": "yandex-gpt-5.1-pro",
    },
    "deepseek": {
        "client": AsyncOpenAI(api_key=DEEPSEEK_API_KEY,
base_url="https://api.deepseek.com/v1/"),
        "model": "deepseek-v3.2",
    },
    "openai": {
        "client": AsyncOpenAI(api_key=OPENAI_API_KEY),
        "model": "gpt-5-mini",
    },
}

def extract_score(text: str) -> int | None:
    match = re.search(r"Результат оценки:\s*(\d{1,3})", text)
    if match:
        return int(match.group(1))
    return None

async def get_score(client: AsyncOpenAI, model: str, prompt: str, semaphore:
asyncio.Semaphore, retries: int = 3) -> int:
    for attempt in range(retries):
        try:
            async with semaphore:
                response = await client.chat.completions.create(
                    model=model,
                    messages=[{"role": "user", "content": prompt}],
                    temperature=0.4,
                    max_tokens=200,
                )
            content = response.choices[0].message.content
            score = extract_score(content)
            if score is not None:
                return score
        except Exception:
            if attempt == retries - 1:
                raise
            await asyncio.sleep(2 ** attempt)
    raise ValueError("Не удалось извлечь оценку после всех попыток")

async def compute_average_score(dataset_path: str, prompt_template: str,
runs_per_model: int = 3) -> float:
    df = pd.read_csv(dataset_path)
    semaphore = asyncio.Semaphore(10)
    tasks = []
    for _, row in df.iterrows():
        prompt = prompt_template.format(vacancy=row["vacancy_text"],
resume=row["resume_text"])
```

```
        for provider in PROVIDERS.values():
            for _ in range(runs_per_model):
                tasks.append(get_score(provider["client"], provider["model"],
prompt, semaphore))
        scores = await asyncio.gather(*tasks, return_exceptions=True)
        total = 0.0
        count = 0
        for res in scores:
            if isinstance(res, int):
                total += res
                count += 1
            else:
                print(f"Ошибка при оценке: {res}", file=sys.stderr)
        return total / count if count > 0 else 0.0
async def main():
    with open("/prompt.txt", "r", encoding="utf-8") as f:
        prompt_template = f.read()
    avg = await compute_average_score("/dataset.csv", prompt_template)
    print(f"{avg:.2f}")
if __name__ == "__main__":
    asyncio.run(main())
```

В результате обработки данных были получены следующие значения средней интегральной оценки по всему датасету: для исходных (сырых) резюме средний балл составил **38,4**; для резюме, оптимизированных платформой Resugen, — **86,3**. Абсолютный прирост составил **47,9** процентных пункта, что соответствует относительному улучшению на **125%** по сравнению с исходным уровнем. Полученные результаты наглядно демонстрируют, что применение платформы Resugen приводит к значительному повышению релевантности резюме требованиям вакансий с точки зрения современных алгоритмических оценщиков.

Таким образом, проведённое тестирование с использованием ансамбля больших языковых моделей подтверждает, что платформа Resugen обеспечивает значимое повышение релевантности резюме требованиям вакансий в парадигме современных интеллектуальных систем отбора.

3.3 Анализ сильных и слабых сторон

Для определения сильных и слабых сторон проекта мы воспользуемся SWOT-анализом. SWOT-анализ предлагаемого ИИ-сервиса по генерации

резюме выявляет внутренние сильные и слабые стороны, а также внешние возможности и угрозы, с учетом специфики модели подписки и регионального рынка труда в России и СНГ. Сильные стороны проекта коренятся в его технологической базе и адаптации к локальным нуждам. Гибридный подход, сочетающий детерминированные алгоритмы для ATS-оптимизации и генеративные модели типа YandexGPT или GigaChat, обеспечивает высокое качество контента, адаптированного под русский язык и HR-практики, что выделяет продукт среди фрагментированных конкурентов, таких как Jobscan или hh.ru конструктор. Интерфейс, ориентированный на простоту и прозрачность (с отчетами о ATS-скорее), снижает барьер входа для пользователей без экспертизы, а модель подписки с Freemium-тарифом стимулирует пробное использование. Ценовая политика (390 руб./месяц) делает сервис доступным для массовой аудитории, а фокус на российской экосистеме (интеграция с hh.ru) укрепляет конкурентные преимущества по сравнению с международными аналогами вроде Kickresume, не учитывающими локальную специфику.

Слабые стороны связаны с внутренними ограничениями стартапа. Зависимость от внешних LLM-провайдеров может привести к нестабильности качества генерации или росту затрат на API-вызовы, особенно при масштабировании до 10 тыс. одновременных пользователей, как указано в нефункциональных требованиях. Интерфейс, хотя и интуитивен, требует доработки для полной адаптивности, а цена подписки может отпугивать пользователей с низкой платежеспособностью, учитывая, что лишь 20-30% от TAM готовы платить за подобные услуги. Команда, ориентированная на IT-разработку, может испытывать дефицит экспертизы в маркетинге, что усложнит достижение SOM в 90-200 млн руб. к третьему году.

Возможности внешней среды обусловлены цифровизацией HR в России и СНГ. Рост внедрения ATS в крупных компаниях (до 85% по данным Deloitte) и эволюция рынка труда, с 11-15 млн активных соискателей

ежегодно, создают спрос на инструменты, повышающие эффективность поиска работы, особенно в секторах ИТ и финансов (5-7 млн потенциальных пользователей). Модель подписки может извлечь выгоду из тенденции к цифровизации, где сервисы вроде CareerLab демонстрируют готовность пользователей платить за премиум-функции, а интеграция с job-порталами открывает каналы для органического роста. Кроме того, развитие отечественных LLM (ruBERT, YaLM) снижает зависимость от импортных технологий, позволяя расширяться на СНГ-рынки с аналогичной лингвистикой.

Угрозы проистекают из динамики конкуренции и изменений на рынке труда. Усиление доминирования hh.ru с его бесплатным конструктором может снизить мотивацию к подписке, а появление новых ИИ-инструментов (например, от Сбера) угрожает дифференциации[26]. Экономическая нестабильность в СНГ, включая колебания спроса на рынке труда, может увеличить меру оттока, подрывая устойчивость модели подписки. Кроме того глобальные тренды, такие как насыщение ИИ-рынка, могут привести к ценовому давлению, снижая ARPA ниже прогнозируемых 780 руб./год.

3.4 Оценка рыночного потенциала

Для объективной оценки коммерческого потенциала проекта и привлечения инвестиций критически важно определить масштаб рынка. Венчурные инвесторы и основатели стартапов используют для этого трехступенчатую модель: TAM (Total Addressable Market) — общий доступный рынок, SAM (Serviceable Addressable Market) — обслуживаемый доступный рынок, и SOM (Serviceable Obtainable Market) — достижимый рынок[27]. Данный анализ позволяет перейти от абстрактных цифр о миллионах соискателей к конкретным, обоснованным количественным целям для бизнеса.

TAM представляет собой максимально возможный объем выручки, которую мог бы генерировать продукт при стопроцентном проникновении на

рынок, без учета ограничений по географии, каналам сбыта или особенностям продукта. Для платформы Resugen целевым является рынок цифровых сервисов для соискателей в Российской Федерации, которые потенциально могут быть заинтересованы в технологичных инструментах для улучшения своих карьерных документов.

В качестве отправной точки можно взять данные о численности экономически активного населения России. По данным Росстата на 2024 год, эта цифра составляет около 69 процентам, а это примерно 100 миллионов человек[28]. Однако далеко не все из них ежегодно занимаются активным поиском работы и готовы платить за соответствующие сервисы. Более релевантной группой являются люди, совершающие карьерные переходы: меняющие работу, ищущие первую работу после окончания учебы или находящиеся в активном поиске. Согласно научно-исследовательскому проекту цифровой аналитики вакансий — Rutrud, доля таких «активных соискателей» в экономически активном населении за февраль 2026 года примерно 700000 человек, исходя из информации об уникальных резюме[29].

Среднее время поиска работы составляет около 2 месяцев[30]. Это означает, что за год через рынок проходит не одна и та же группа людей, а примерно 6 последовательных «слоёв» соискателей. Вычисляя по Формуле 3.3.1, получается примерно 4200000 уникальных соискателей в год.

$$TAM(\text{чел.}/\text{год}) = \text{уникальные соискатели в месяц} \cdot \left(\frac{12 \text{ месяцев}}{\text{Средний срок поиска}} \right) \quad (3.3.1)$$

В денежном выражении, если предположить среднегодовую выручку с одного платящего пользователя (ARPA) на уровне 780 рублей (эквивалент подписки 390 руб./мес. с учетом среднего времени поиска работы в 2 месяца), то общий потенциальный объем рынка (TAM) составляет примерно 3.28 миллиардов рублей в год. Эта оценка является теоретическим

максимумом и показывает верхнюю границу возможностей бизнеса в национальном масштабе.

SAM представляет собой сегмент TAM, на который компания может реально выйти со своим продуктом, учитывая его особенности, ценовую политику, каналы дистрибуции и текущие операционные ограничения. Это рынок, который географически и концептуально достижим для стартапа на текущем этапе.

Первым фильтром для сужения TAM до SAM является технологический и продуктовый фокус. Платформа Resugen изначально заточена под глубокий анализ и оптимизацию для ATS (Applicant Tracking Systems). Следовательно, ее основная ценность очевидна для соискателей, которые целенаправленно откликаются на вакансии в компаниях, использующих подобные системы. Это, в первую очередь, средний и крупный бизнес, международные корпорации, а также рекрутинговые агентства. Соискатели, ищущие работу в малом бизнесе, на фрилансе или через прямые знакомства, в меньшей степени являются целевой аудиторией на старте. Это сокращает целевую группу примерно на 30-40%.

Второй фильтр — лингвистический и региональный. На этапе MVP и в первые годы развития продукт будет полностью заточен под русский язык и специфику российского рынка труда (особенности hh.ru, 1С-платформ). Это исключает из SAM русскоязычных соискателей за рубежом, ориентированных на иностранные рынки, и сужает фокус до территории РФ.

Третий фильтр — каналы привлечения и осведомленность. Стартап не сможет мгновенно охватить всех потенциальных пользователей в силу ограниченности маркетингового бюджета. Первоначально привлечение будет идти через цифровые каналы (поиск, контекстная реклама, контент-маркетинг), которые наиболее эффективны для технологически продвинутой аудитории.

С учетом этих ограничений, SAM можно оценить как 20-30% от TAM. Это аудитория, которая не только теоретически нуждается в продукте, но и

может узнать о нем через имеющиеся каналы коммуникации. В денежном выражении при том же ARPA в 780 рублей, SAM составляет 0.66 — 0.98 миллиардов рублей в год. Эта цифра демонстрирует реалистичный, но все еще амбициозный рыночный потенциал для компании, которая сможет стать лидером в своей нише.

SOM — это часть SAM, которую компания реально планирует захватить в обозримой перспективе, обычно в первые 3-5 лет работы. Это наиболее консервативная и прагматичная оценка, которая учитывает наличие конкурентов, ресурсные ограничения стартапа, скорость роста и стратегию выхода на рынок. SOM является основой для построения финансовых прогнозов и постановки конкретных бизнес-целей.

Расчет SOM начинается с анализа доли рынка, которую новый игрок может разумно отвоевать. Несмотря на выявленную нишу, на рынке присутствуют конкуренты: бесплатный конструктор hh.ru (доминирующий игрок с максимальным охватом), платные нишевые сервисы вроде CareerLab, а также общая тенденция к использованию коммерческих LLM (YandexGPT, GigaChat и другие). Ключевой задачей является переключение пользователей с бесплатного, но неинтеллектуального решения hh.ru на платный, но дающий измеримую пользу продукт. В условиях SaaS-рынка захват 5-10% от SAM за первые 3-4 года считается отличным результатом для умелого стартапа с сильным продуктом.

Далее учитывается поэтапность выхода. В первый год фокус будет на узкой, но высокоплатежеспособной группе — IT-специалистах. Во второй год планируется расширение на смежные области (финансы, инженерия, digital-маркетинг). Только к третьему-четвертому году можно говорить о широком охвате всей аудитории SAM.

Исходя из этого, реалистичной целью на третий год работы можно считать привлечение 2-3% от SAM. В абсолютных цифрах это 20 000 — 30 000 платящих пользователей. При ARPA в 780 рублей, целевой объем SOM на третий год составляет 15.6 – 23.4 миллионов рублей годовой выручки

(ARR). Для первого года работы цель будет скромнее — привлечение первых 3000-4500 платящих пользователей и достижение выручки в 2.34 – 3.51 миллионов рублей.

На Рисунке 3.3.1 предоставлены диаграммы по результатам проведенного анализа.

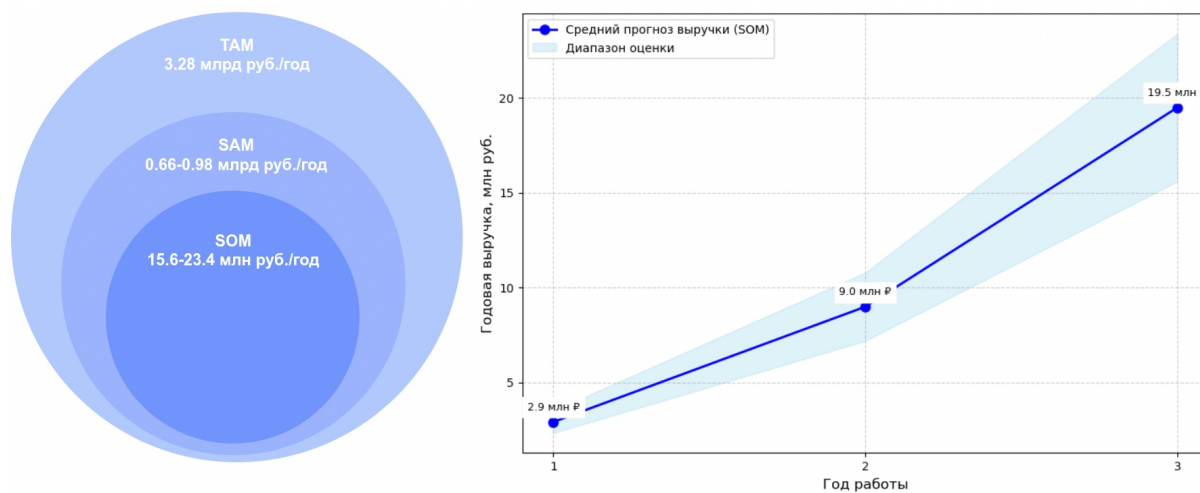


Рисунок 3.3.1 — Результаты анализа TAM, SAM, SOM

ЗАКЛЮЧЕНИЕ

В результате выполнения дипломной работы была достигнута поставленная цель: разработана архитектура и создан минимально жизнеспособный продукт (MVP) интеллектуальной веб-платформы «Resugen», выступающей в роли «цифрового карьерного ассистента». Решение позволяет соискателям адаптировать свои резюме к современным алгоритмическим системам отбора персонала за счёт автоматизированного анализа требований вакансий и интеллектуальной генерации структурированного контента. В ходе работы решены все сформулированные задачи:

1. Изучены международные и российские сервисы для создания и оптимизации резюме (включая конструкторы, ATS-ориентированные инструменты и платформы с ИИ-ассистентами). Выявлены их ключевые недостатки: отсутствие глубокой семантической привязки к конкретной вакансии, слабая поддержка совместимости с ATS и недостаточная прозрачность критериев оценки.
2. Детально описаны сценарии использования платформы: загрузка вакансии, анализ резюме, рекомендации по улучшению, генерация адаптированной версии документа. Определены нефункциональные требования (производительность, масштабируемость, безопасность данных, удобство интерфейса), что обеспечило системный подход к разработке.
3. Разработана микросервисная архитектура, включающая сервисы бизнес-процесса и управления пользователями.
4. Реализованы дополнительные сервисы бизнес-процесса и фронтенд-часть.
5. Разработана модель Freemium с базовым бесплатным функционалом и платными подписками и произведена оценка рыночного потенциала

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. The Evolution of Applicant Tracking System: A Historical Perspective // OnBlick URL: <https://www.onblick.com/blogs/the-evolution-of-applicant-tracking-system-a-historical-perspective> (дата обращения: 18.04.2026).
2. Naoki Otani, Nikita Bhutani, Estevam Hruschka, Natural Language Processing for Human Resources: A Survey // Aclanthology. - 2025. - С. 583–597.
3. АТС система для ускорения подбора персонала: как технологии меняют рекрутинг в 2026 году // VC.ru URL: <https://vc.ru/hr/2787930-ats-sistema-dlya-podbora-personala> (дата обращения: 18.04.2026).
4. ИИ в скрининге резюме и подборе кандидатов: полный гид // 1SELLER URL: <https://1seller.ru/ii-v-skrininge-rezyume-i-podbore-kandidatov-polnyj-gid> (дата обращения: 18.04.2026).
5. Enhancing recruitment efficiency: An advanced Applicant Tracking System (ATS) // ResearchGate URL: https://www.researchgate.net/publication/382099903_Enhancing_recruitment_efficiency_An_advanced_Applicant_Tracking_System_ATS (дата обращения: 18.04.2026).
6. ИИ против горы резюме: как научить модель ранжировать кандидатов на данных вашей компании // HR Завтра URL: <https://naimee.ai/hrzavtra/ii-rezyume-obuchenie-modeli-ranzhirovat-kandidatov> (дата обращения: 18.04.2026).
7. Лучшие AI-решения для HR и подбора персонала в 2025 году: полный гид по технологиям искусственного интеллекта // VC.ru URL: <https://vc.ru/hr/2689003-ai-resheniya-dlya-hr-i-podbora-personala> (дата обращения: 18.04.2026).
8. How AI Chatbots Are Transforming Recruitment in 2026 // assess candidates URL: <https://www.assesscandidates.com/ai-chatbots-for-recruitment/> (дата обращения: 18.04.2026).

9. Suraj Francis Noronha, Aruna Doreen Manezes, Poornima shetty, V. S. Shrishma Rao Evolving Human Resource Management: Admitting the Power of Artificial Intelligence // ATLANTIS PRESS. - 2025. - С. 270-281.
10. Jane Castleman, Zeyu Shen, Blossom Metevier, Max Springer, Aleksandra Korolova Measuring Validity in LLM-based Resume Screening // ATLANTIS PRESS. - 2026
11. 15 лучших генераторов резюме на основе ИИ в 2026 году (обзор бесплатных и платных инструментов) // Autoppt URL: <https://autoppt.com/ru/blog/best-ai-resume-generators/> (дата обращения: 18.04.2026).
12. ИИ отсеивает большую часть соискателей: 75% резюме никогда не попадают к рекрутеру // ОНТ URL: <https://ont.by/ru/society-ru/view/ii-otseivaet-bolshuju-chast-soiskatelej-75-rezjume-nikogda-ne-popadajut-k-rekruteru-315277-2026> (дата обращения: 18.04.2026).
13. Что делать, если меня не берут на работу из-за ИИ-алгоритмов? // Т-Ж URL: <https://t-j.ru/ii-v-naime/> (дата обращения: 18.04.2026).
14. Системы скрининга резюме. Как адаптировать резюме к ATS чтобы получить работу? // VC.ru URL: <https://vc.ru/hr/1356442-sistemy-skrininga-rezyume-kak-adaptirovat-rezyume-k-ats-chtoby-poluchit-rabotu> (дата обращения: 18.04.2026).(B6) Мнение: соискателям нужно прекратить использовать ИИ для резюме и тестовых заданий // Хабр URL: <https://habr.com/ru/news/1002922/> (дата обращения: 18.04.2026).
15. Создание резюме для ИИ: ключевые моменты и роль помощника по подбору персонала ONO AI // ONO URL: <https://onenewone.com/blog-ru/Создание-резюме-для-ИИ-ключевые-момен> (дата обращения: 18.04.2026).
16. Мнение: соискателям нужно прекратить использовать ИИ для резюме и тестовых заданий // Хабр URL: <https://habr.com/ru/news/1002922/> (дата обращения: 18.04.2026).

17. ATS для HR в 2025. Что происходит в России и мире — и к чему готовиться бизнесу // HR Завтра URL: <https://naimee.ai/hrzavtra/ats-dlya-hr-v-2025-perspektivy-i-trendy> (дата обращения: 18.04.2026).
18. Snehal Suryawanshi, Prajwal Mali, Sarthak Rasal Resume parsing and job recommendation using NLP and Machine learning // IJRTI. - 2025. - № 2456-3315
19. Екатерина Субботина, Александр Анисимов, Мария Засканова Открываем новые звезды автоматизации рекрутмента на российском рынке вместе с ТеДо // Хантфлоу. - 2025
20. Лучшие нейросети для создания резюме в 2025 году // VC.ru URL: <https://vc.ru/hr/2325297-luchshie-neurosseti-dlya-sozdaniya-rezyume> (дата обращения: 18.04.2026).
21. ТОП-8 лучших сервисов для создания профессионального резюме в 2025 году // VC.ru URL: <https://vc.ru/hr/2614507-luchshie-servisy-dlya-sozdaniya-rezyume> (дата обращения: 18.04.2026).
22. Jiawei Gu, Xuhui Jiang, Zhichao Shi, Hexiang Tan, Xuehao Zhai, Chengjin Xu, Wei Li, Yinghan Shen, Shengjie Ma, Honghao Liu, Saizhuo Wang, Kun Zhang, Zhouchi Lin, Bowen Zhang, Lionel Ni, Wen Gao, Yuanzhuo Wang, Jian Guo A survey on LLM-as-a-judge // ScienceDirect. - 2025
23. Frank P.-W. Lo, Jianing Qiu, Zeyu Wang, Haibao Yu, Yeming Chen, Gao Zhang, Benny Lo AI Hiring with LLMs: A Context-Aware and Explainable Multi-Agent Framework for Resume Screening // arxiv. - 2025
24. Ryan Lail An Empirical Investigation of Practical LLM-as-a-Judge Improvement Techniques on RewardBench 2 // arxiv. - 2026
25. Vidhi Agrawal From Freemium To Enterprise: Rethinking SaaS Pricing // arxiv. - 2025
26. ФАС признала hh.ru доминирующим на рынке онлайн-рекрутинга // NEW-RETAIL URL: https://new-retail.ru/novosti/retail/fas_priznala_hh_ru_dominiruyushchim_na_rynke_onlayn_rekrutinga/ (дата обращения: 18.04.2026).

27. Определить TAM SAM SOM // Podbiratel URL:
<https://podbiratel.com/pm/market-ideas-analysis/tam-sam-som> (дата обращения: 18.04.2026).
28. Уровень экономически активного населения в Российской Федерации // iMonitoring URL: <https://www.iminfin.ru/news/584-uroven-ekonomicheski-aktivnogo-naseleniya-v-rossijskoj-federatsii> (дата обращения: 18.04.2026).
29. Краткий отчет по рынку труда за февраль 2026 // rutrud.com URL:
<https://rutrud.com/2026-02/> (дата обращения: 18.04.2026).
30. Какой срок поиска работы считается нормальным? // Т-Ж URL:
<https://t-j.ru/job-search-duration/> (дата обращения: 18.04.2026).